CM 9 : Réseaux de neurones convolutifs pour le texte

Cours repris de Voita (2025)

Ljudmila PETKOVIĆ
Semestre 2, 2025-2026
3 avril 2026

Sorbonne Université
Licence « Sciences du Langage » (L3 ScLan)
UFR Sociologie et Informatique pour les Sciences Humaines

An absolutely great movie! I watched the premiere with my friends.

The movie about cats was absolutely great, and the cats were cute.

The movie is about cats running around, and it is absolutely great.

If a clue is very informative, maybe we don't care much where in a text it appears?

Page source : lena-voita.github.io/nlp_course/models/convolutional.html

Vue d'ensemble et portée

- modèles convolutionnels et configurations adaptées aux tâches de TAL
- cours principal : source
- réseaux de neurones convolutifs → CNN

Feuille de route : intuition → briques de base → classification de texte → modélisation du langage → interprétabilité

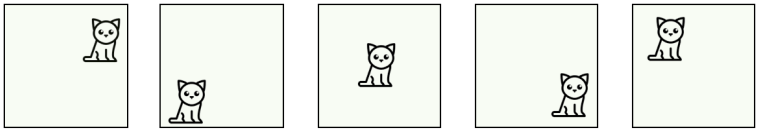


Paramètres

- taille du noyau (*kernel*)
- pas (*stride*)
- remplissage (*padding*)
- biais (*bias*)
- sous-échantillonnage (*k-max pooling*)

Convolutions pour les images et invariance par translation

Classification d'images



We don't care where the cat is, we care that it is somewhere.

Then why don't we process all these cats similarly?

- origine des CNN : vision par ordinateur (*computer vision*) [1]
- Imaginons que nous voulions classer une image en plusieurs catégories, p. ex. chat, chien, avion, etc.
- S'il y a un chat dans l'image, on veut surtout savoir qu'il est *là quelque part*, pas exactement où il apparaît.
- Les CNN appliquent cette opération locale à de petites régions et apprennent des motifs utiles pour la classification.
- Chaque opération recherche une correspondance avec un motif, et le réseau apprend quels motifs sont utiles.
- Avec de nombreuses couches, les motifs appris vont de lignes simples à des objets entiers (p. ex. chat ou chien).
- Cette propriété s'appelle l'invariance par translation : un décalage spatial ne devrait pas changer la décision de classe.

Voir l'animation illustrant ce principe.

1. sous-domaine de l'IA qui permet aux machines d'analyser et d'interpréter des données visuelles (images, vidéos) (IBM, *s.d.*).

Convolutions pour le texte

An absolutely great movie! I watched the premiere with my friends.

The movie about cats was absolutely great, and the cats were cute.

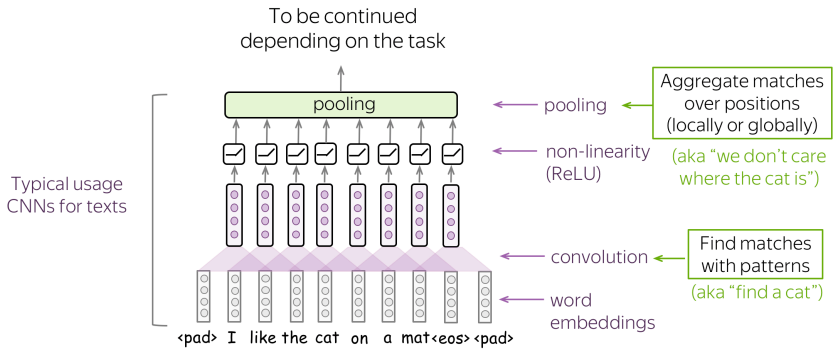
The movie is about cats running around, and it is absolutely great.

If a clue is very informative, maybe we don't care much where in a text it appears?

Quant aux textes, l'intuition issue de l'image ne se transpose pas littéralement, car :

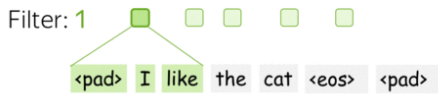
- déplacer arbitrairement un segment change le sens.
- Mais dans des tâches comme la classification de sentiment, certaines expressions locales sont très informatives :
  - it's been great
  - bored to death
  - absolutely amazing
  - the best ever
- Pour ce type d'indices, l'endroit exact où ils apparaissent dans le document compte souvent peu.
- La détection de motifs locaux suivie d'une agrégation sur les positions convient aux CNN pour le texte.

Modèle typique : blocs (couches) convolution + *pooling*



- La convolution repère les correspondances avec des motifs utiles (≈ tête du chat dans l'animation).
- Une non-linéarité, en général ReLU, est appliquée après la convolution.
- Le pooling agrège ces correspondances sur les positions, localement ou globalement.
- Pour le texte, une couche convolutionnelle est généralement appliquée aux embeddings de mots,
- puis suivie d'une non-linéarité et d'un pooling.

Briques de base : convolution

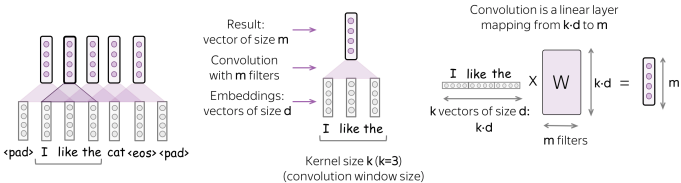


Convolution d'image : un filtre glisse sur deux dimensions : la largeur et la hauteur (source).

Convolution de texte : la séquence est unidimensionnelle, donc les filtres ne glissent que sur les positions des tokens (source).

- En vision par ordinateur, la convolution utilise une fenêtre glissante sur une entrée 2D (largeur et hauteur).
  - image du bas : image d'entrée ; celle du haut : sortie du filtre
- Pour le texte, la convolution est 1D : la même opération locale est appliquée à des fenêtres successives de tokens.
- Une couche contient généralement plusieurs filtres, chacun chargé d'un motif différent.

La convolution est une opération linéaire appliquée à chaque fenêtre



Une convolution n'est qu'une couche linéaire partagée sur des fenêtres locales, généralement suivie d'une non-linéarité.

Supposons des représentations d'entrée $(x_1, \ldots, x_n)$, où $x_i \in \mathbb{R}^d$.

- d : canaux d'entrée (taille des embeddings)
- k : taille du noyau (longueur de la fenêtre convolutive, ici $k = 3$)
- m : canaux de sortie (nombre de filtres convolutifs, soit le nombre de canaux produits par la convolution)

Une convolution est une couche linéaire $W \in \mathbb{R}^{(k \cdot d) \times m}$. Pour une fenêtre de taille k, $(x_i, \ldots, x_{i+k-1})$, la convolution prend la concaténation de ces vecteurs :

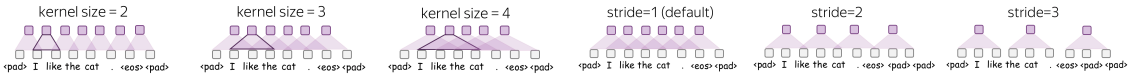
$$u_i = [x_i, \ldots, x_{i+k-1}] \in \mathbb{R}^{k \cdot d}$$

puis on applique la même matrice

$$W \in \mathbb{R}^{(k \cdot d) \times m}, \qquad F_i = u_i W.$$

Une convolution parcourt une entrée avec une fenêtre glissante et applique la même transformation linéaire à chaque fenêtre.

Paramètres I : taille du noyau et pas



Taille du noyau : jusqu'où regarder

C'est le nombre de tokens d'entrée inspectés par la convolution à chaque étape. Pour le texte, les valeurs typiques sont 2–5.

Pas : de combien se déplacer

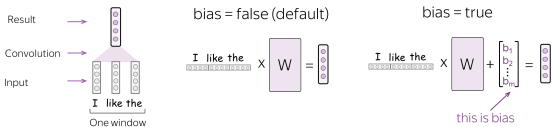
Le pas indique de combien le filtre se décale à chaque étape. Un pas de 1 signifie avancer d'un token à chaque fois pour les textes (ou d'un pixel pour les images).

Paramètres II : remplissage et biais



Remplissage : ajouter des vecteurs nuls des deux côtés

Le remplissage modifie les fenêtres possibles près des bords de la séquence. Utile lorsque le pas > 1.



Biais : le terme de biais dans l'opération linéaire de la convolution.

Sans biais, l'opération n'est qu'une multiplication matricielle ; avec biais, chaque canal de sortie reçoit un décalage supplémentaire.

Intuition : chaque filtre extrait une caractéristique

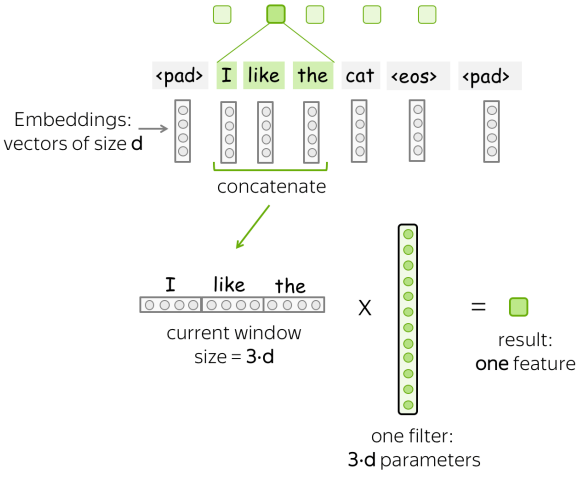
- Un filtre = un extracteur de caractéristiques.
- Pour la fenêtre courante

$$u_i = [x_i, \ldots, x_{i+k-1}] \in \mathbb{R}^{k \cdot d},$$

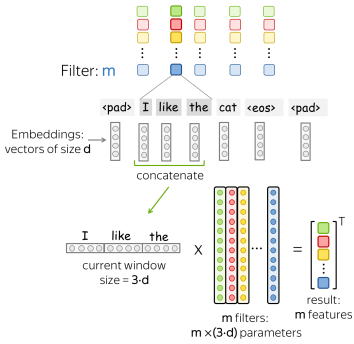
un filtre $f \in \mathbb{R}^{k \cdot d}$ calcule le produit scalaire

$$F_i^{(f)} = (f, u_i).$$

- Le nombre $F_i^{(f)}$ (la « caractéristique » extraite) est le résultat de l'application du filtre f à la fenêtre $(x_i, \ldots, x_{i+k-1})$.
- Un filtre possède $k \cdot d$ paramètres.



m filtres : m extracteurs de caractéristiques en parallèle

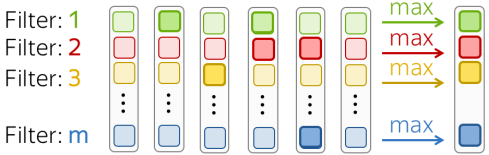


Différents filtres lisent le même texte et se spécialisent dans des motifs différents (animation).

- Un filtre extrait une seule caractéristique.
- Plusieurs caractéristiques $\rightarrow$ plusieurs filtres
- Chaque filtre lit un texte d'entrée
- et en extrait une caractéristique différente.
- nb de filtres = nb de caractéristiques de sortie
- Tous les filtres s'exécutent en parallèle.
- taille de la couche convolutive : $(k \cdot d) \times m$

L'application linéaire partagée a la forme $(k \cdot d) \times m$, ce qui permet à différents filtres d'extraire des caractéristiques locales différentes à partir d'une même fenêtre.

Briques de base : pooling

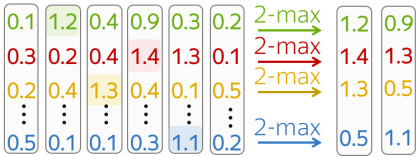


Max pooling: maximum for each dimension (feature)



- Après que la convolution a extrait *m* caractéristiques pour chaque fenêtre,
- une couche de pooling résume ces caractéristiques sur une certaine région.
- Le pooling réduit la dimension de l'entrée et donc le nombre de paramètres dans les couches suivantes.
- Le max-pooling prend le maximum de chaque caractéristique ; le mean-pooling en prend la moyenne.
- Chaque caractéristique est activée lorsqu'elle détecte un motif :
  - un motif visuel dans une image (ligne, texture, empreinte de patte de chat, etc.)
  - ou un motif textuel (par exemple, une phrase).
- Après le pooling, on obtient un vecteur indiquant quels motifs étaient présents dans l'entrée.

k-max pooling



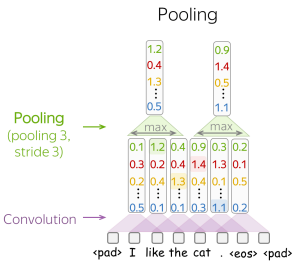
k-max pooling:
k highest values in their original order

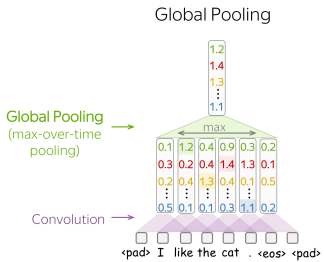
- Le k-max pooling généralise le max-pooling.
- Au lieu d'un seul maximum, il conserve les k plus grandes valeurs.
- Leur ordre d'origine est conservé.
- Il est utile lorsque le modèle doit se souvenir *du nombre de fois* où un motif a été trouvé.

Pooling, pas et pooling global



Le pooling s'applique lui aussi à des fenêtres de plusieurs éléments et possède un paramètre de pas. Pour utiliser des fenêtres non recouvrantes, fixez pas = taille du pool.

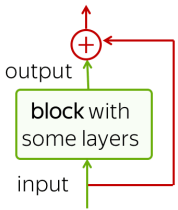




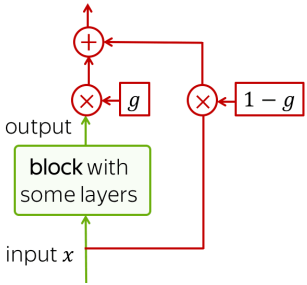
- Le pooling est appliqué indépendamment aux caractéristiques de chaque fenêtre.
- Le pooling global résume l'entrée entière.
- Pour les textes, on utilise souvent un pooling global pour obtenir un vecteur unique représentant l'ensemble du texte.
- Ce pooling global est appelé max-over-time pooling, où l'axe du « temps » va du premier au dernier token d'entrée.
- Chaque caractéristique « s'active » lorsqu'elle repère un certain motif :
  - un motif visuel dans une image (ligne, texture, patte de chat, etc.)
  - ou un motif textuel (par exemple, une expression).
- Après une opération de pooling, on obtient un vecteur indiquant lesquels de ces motifs sont apparus dans l'entrée.

Briques de base : connexions résiduelles

TL ;DR : Entraîner facilement les réseaux profonds



Residual connection: add a block's input to its output



Highway connection: gated sum of a block's input and output

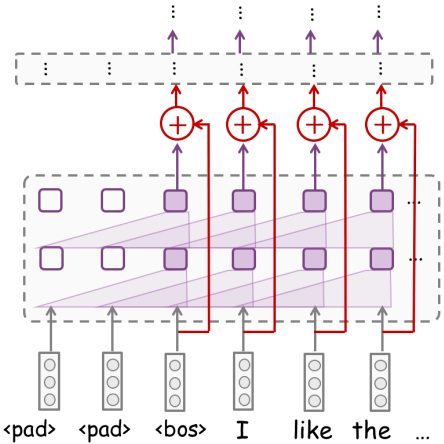
$$g = \sigma(Wx + b)$$

Gate: because of σ, its values are in (0, 1)

- Pour traiter des contextes plus longs, il faut souvent de nombreuses couches convolutionnelles.
- Des empilements profonds compliquent la propagation du gradient.
- Pour éviter cela :
  - les connexions résiduelles ajoutent simplement l'entrée d'un bloc à sa sortie et facilitent la circulation du gradient.
  - les connexions Highway utilisent une somme à porte ($\approx$ portes des LSTM, qui contrôlent l'information transmise).

Blocs résiduels dans les réseaux convolutionnels

- Les connexions résiduelles sont généralement placées autour de blocs comportant plusieurs couches convolutionnelles.
- Un réseau peut empiler plusieurs blocs résiduels de ce type.
- Plus la tâche exige de contexte, plus il faut de profondeur — et plus ces raccourcis deviennent importants.



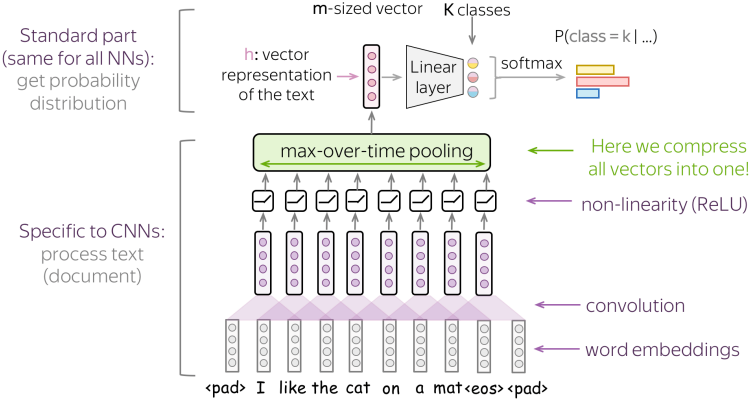
more blocks

residual connection around the block

block with 2 convolutional layers

Tâche spécifique : classification de texte

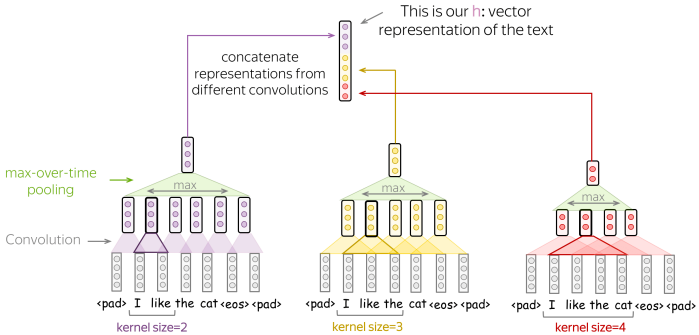
Voir le cours dédié



Nous avons besoin de construire un modèle convolutif qui peut produire un vecteur de taille fixe pour des textes de longueurs différentes.

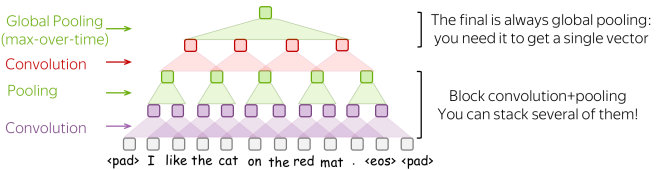
- L'opération clé est le pooling global (souvent max-over-time pooling).
- Il compresse tous les vecteurs positionnels en une seule représentation du texte.
- Ce vecteur est ensuite envoyé à un classifieur.

Classification de texte : plusieurs tailles de noyau



- Utiliser plusieurs tailles de noyau en parallèle au lieu de se limiter à une seule longueur de fenêtre.
- Après chaque convolution, appliquer une non-linéarité puis un pooling global.
- Concaténer les sorties agrégées pour construire la représentation du texte.
- Cette recette correspond à la configuration classique de Kim (2014) et a inspiré de nombreux travaux ultérieurs.

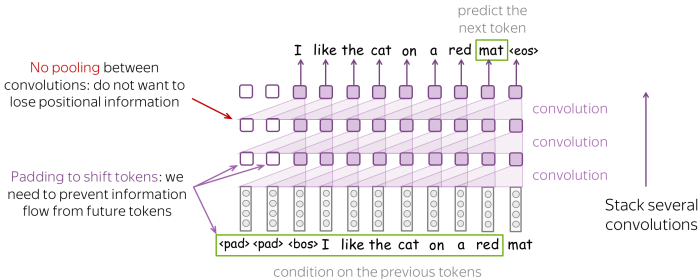
Classification de texte : empiler plusieurs blocs de convolution + pooling



- Au lieu d'une seule couche, empiler plusieurs blocs convolution + pooling.
- L'étape finale doit toujours utiliser un pooling global pour produire un vecteur unique.
- C'est particulièrement utile pour des entrées très longues :
  - p. ex., si le modèle est au niveau du caractère (et non au niveau du mot).
- Cette idée apparaît dans Zhang *et al.*, 2015.

Tâche spécifique : modélisation du langage

Voir le cours **dédié**

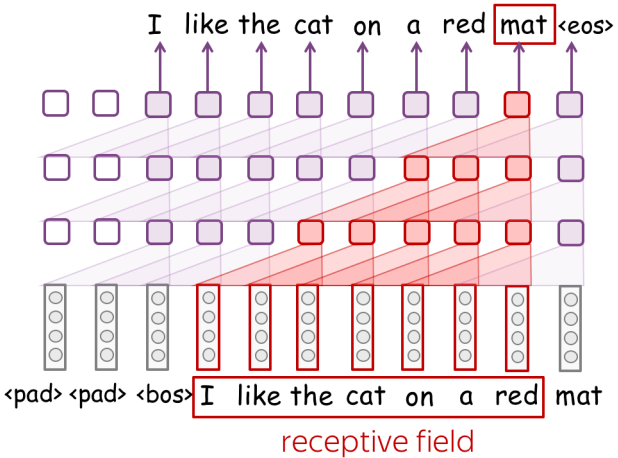


- Empêcher les fuites vers le futur : un modèle de langue gauche-droite ne doit utiliser que les tokens précédents.
- Conserver l'information positionnelle : contrairement à la classification, la position des tokens compte, donc il faut éviter le pooling ou l'utiliser avec beaucoup de précaution.
- Utiliser des connexions résiduelles : les contextes longs exigent des empilements profonds, et ces empilements sont difficiles à entraîner sans raccourcis.

Modélisation du langage : champ récepteur

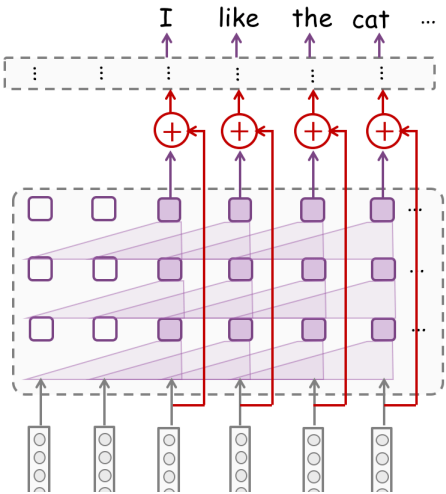
- Sans pooling global, un modèle convolutionnel a nécessairement un contexte de taille fixe.
- Cela peut sembler problématique : c'est précisément ce que l'on reprochait aux modèles en n-grammes.
- Dans les modèles en n-grammes, le contexte est généralement très court (souvent 1 à 4 tokens).
- Dans les modèles convolutionnels, le contexte peut être bien plus long.
- Avec seulement 3 couches convolutionnelles et une taille de noyau de 3, le contexte couvre déjà 7 tokens.
- En empilant davantage de couches, on peut obtenir un contexte beaucoup plus large.

Intuition : un CNN a un contexte fixe, mais potentiellement bien plus large que celui d'un modèle en n-grammes.



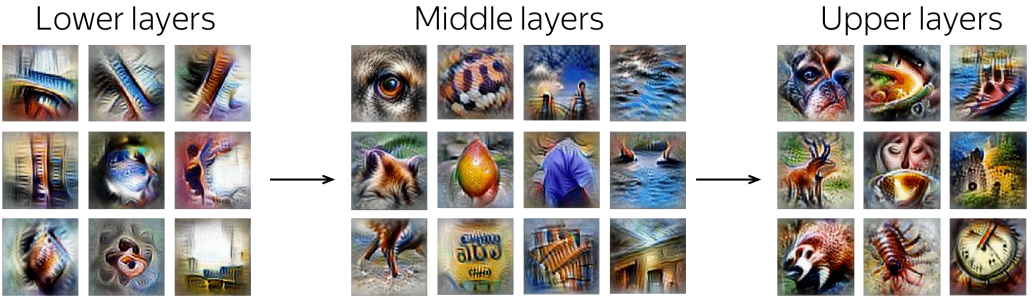
Modélisation du langage : blocs résiduels pour des CNN profonds

- En empilant de nombreuses couches, l'entraînement du réseau peut devenir difficile.
- On peut alors utiliser des connexions résiduelles.
- En général, on place ces connexions autour de blocs contenant plusieurs couches.
- Un réseau peut empiler plusieurs blocs de ce type.
- Cette profondeur est utile pour obtenir un champ récepteur suffisamment large.



Qu'apprennent les convolutions ?

Analyse des filtres convolutionnels en vision par ordinateur



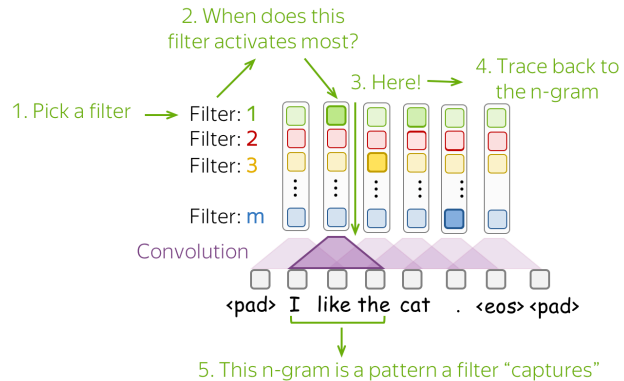
(source).

- Pour les images, les couches basses capturent des motifs visuels simples, comme des lignes ou des cercles.
- Les couches intermédiaires et supérieures capturent des combinaisons de plus en plus riches :
  - des objets entiers : animaux, personnes, scènes.
- C'est l'un des exemples les plus nets d'apprentissage hiérarchique de caractéristiques dans les réseaux profonds.

Analyse en classification de texte : filtres comme détecteurs de n-grammes

Principaux constats pour les CNN textuels :

- Les filtres convolutionnels agissent comme des détecteurs de n-grammes.
- Un filtre peut capturer une ou plusieurs familles de n-grammes étroitement liées.
- Les filtres n'ont pas besoin d'être homogènes : un même filtre peut s'activer pour des familles distinctes.
- Le max-pooling induit un effet de seuil, si bien que les activations faibles sont ignorées.
- Jacovi *et al.* (2020) rapportent qu'environ 40 % des n-grammes agrégés peuvent être supprimés sans perte de performance.
- La manière la plus simple de comprendre ce qu'un réseau capture consiste à regarder quels motifs activent ses neurones.
- Pour les convolutions, on choisit un filtre et on cherche quels n-grammes activent le plus ce filtre.



Analyse en classification de texte : meilleurs n-grammes par filtre

filter	Top n-gram	Score
1	poorly designed junk	7.31
2	simply would not	5.75
3	a minor drawback	6.11
4	still working perfect	6.42
5	absolutely gorgeous .	5.36
6	one little hitch	5.72
7	utterly useless .	6.33
8	deserves four stars	5.56
9	a mediocre product	6.91

Top n-grams for filter 4

		Score
1	still working perfect	6.42
2	works - perfect	5.78
3	isolation proves invaluable	5.61
4	still near perfect	5.6
5	still working great	5.45
6	works as good	5.44
7	still holding strong	5.37

A filter activates for a family of n-grams with similar meaning

- Les n-grammes qui activent le plus un filtre révèlent ce qu'il a appris.
- L'exemple mis en évidence s'active pour une famille d'expressions sémantiquement proches.
- C'est un indice fort que les filtres de CNN textuels capturent des motifs locaux cohérents.

Voir Jacovi *et al.* (2020).

Analyse en modélisation du langage : familles de phrases apprises

have until **nov.**
operation since **aug.**
quarter ended **sept.**
terrible tuesday **oct.**
even before **june**

amr chairman **robert**
chief economist **john**
chicago investor **william**
exchange chairman **john**
texas billionaire **robert**

as little as
of more than
as high as
as much as
as low as

facilities will **substantially**
which would **substantially**
dean witter **actually**
we 'll **probably**
you should **really**

no matter **how**
are afraid **how**
question is **how**
remaining are **how**
to say **how**

a merc **spokesman**
a company **spokesman**
a boeing **spokesman**
a fidelity **spokesman**
a quotron **spokeswoman**

would **allow** the
does **allow** the
still **expect** ford
warrant **allows** the
funds **allow** investors

more evident **among**
a dispute **among**
bargain-hunting **among**
growing fear **among**
paintings listed **among**

- Pour un CNN, Pham *et al.* (2016) alimentent le modèle avec les données de développement et recherchent les n-grammes qui activent le plus fréquemment un filtre donné.
- Les filtres cherchent des types de phrases qui peuvent être poursuivies de façon similaire.
- Un noyau peut s'activer sur des phrases se terminant par des mois, sur des noms, sur des tournures de porte-parole ou sur des patrons comparatifs comme as . . . as.

Références et crédits

- Page source du cours : Réseaux de neurones convolutifs pour le texte
- Dépôt original du cours : lena-voita.github.io
- Illustrations de l'arithmétique des convolutions : vdumoulin/conv_arithmetic
- Srivastava *et al.* (2015) : Highway Networks
- Kim (2014) : CNNs for Sentence Classification
- Zhang *et al.* (2015) : Character-level ConvNets for Text Classification
- Jacovi *et al.* (2020) : Understanding CNNs for Text Classification
- Pham *et al.* (2016) : Convolutional Neural Network Language Models
- Exemples d'Activation Atlas : distill.pub

Natural Language Processing with Deep Learning

CS224N/Ling284



Lecture 6:
Language Models and
Recurrent Neural Networks

Abigail See

Overview

Today we will:

- Introduce a new NLP task
  - **Language Modeling**



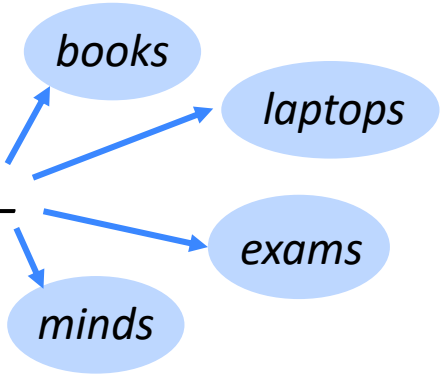
- Introduce a new family of neural networks
  - **Recurrent Neural Networks (RNNs)**

These are two of the most important ideas for the rest of the class!

Language Modeling

- **Language Modeling** is the task of predicting what word comes next.

the students opened their _____



- More formally: given a sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$, compute the probability distribution of the next word $\boldsymbol{x}^{(t+1)}$:

$$P(\boldsymbol{x}^{(t+1)} | \ \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$$

where $\boldsymbol{x}^{(t+1)}$ can be any word in the vocabulary $V = \{\boldsymbol{w}_1, \ldots, \boldsymbol{w}_{|V|}\}$

- A system that does this is called a **Language Model**.

Language Modeling

- You can also think of a Language Model as a system that assigns probability to a piece of text.

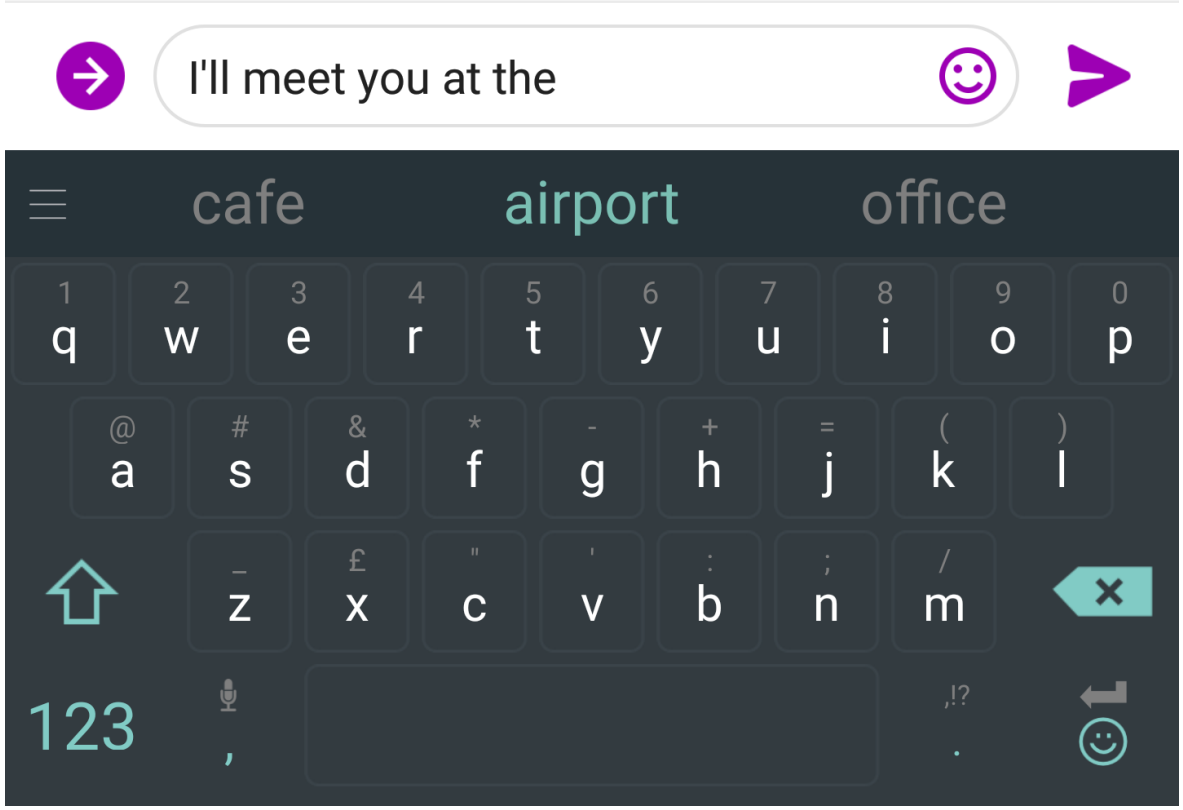
- For example, if we have some text $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, then the probability of this text (according to the Language Model) is:

$$P(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = P(\boldsymbol{x}^{(1)}) \times P(\boldsymbol{x}^{(2)} | \boldsymbol{x}^{(1)}) \times \cdots \times P(\boldsymbol{x}^{(T)} | \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)})$$

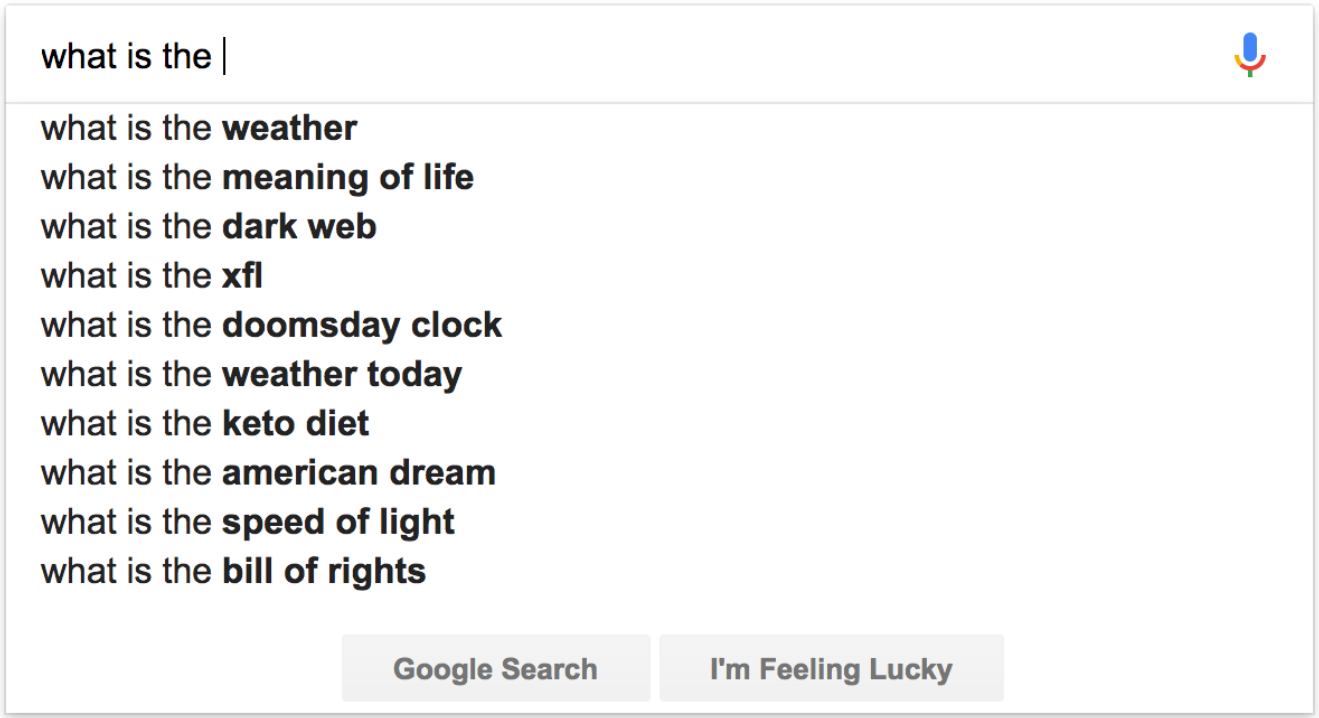
$$= \prod_{t=1}^{T} P(\boldsymbol{x}^{(t)} | \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)})$$

This is what our LM provides

You use Language Models every day!



You use Language Models every day!



n-gram Language Models

the students opened their _____

- **Question**: How to learn a Language Model?
- **Answer** (pre- Deep Learning): learn a *n*-gram Language Model!

- Definition: A *n*-gram is a chunk of *n* consecutive words.
  - unigrams: “the”, “students”, “opened”, ”their”
  - bigrams: “the students”, “students opened”, “opened their”
  - trigrams: “the students opened”, “students opened their”
  - 4-grams: “the students opened their”

- Idea: Collect statistics about how frequent different n-grams are, and use these to predict next word.

n-gram Language Models

- First we make a simplifying assumption: $\boldsymbol{x}^{(t+1)}$ depends only on the preceding *n-1* words.

$$P(\boldsymbol{x}^{(t+1)}|\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(1)}) = P(\boldsymbol{x}^{(t+1)}|\overbrace{\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)}}^{n-1 \text{ words}}) \quad \text{(assumption)}$$

prob of a n-gram

prob of a (n-1)-gram

$$= \frac{P(\boldsymbol{x}^{(t+1)},\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}{P(\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})} \quad \text{(definition of conditional prob)}$$

- **Question:** How do we get these *n*-gram and (*n*-1)-gram probabilities?
- **Answer:** By counting them in some large corpus of text!

$$\approx \frac{\text{count}(\boldsymbol{x}^{(t+1)},\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}{\text{count}(\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})} \quad \text{(statistical approximation)}$$

n-gram Language Models: Example

Suppose we are learning a 4-gram Language Model.

~~*as the proctor started the clock, the*~~ *students opened their* _____

~~as the proctor started the clock, the~~ — discard

students opened their — condition on this

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

- "students opened their" occurred 1000 times
- "students opened their books" occurred 400 times
  - → P(books | students opened their) = 0.4
- "students opened their exams" occurred 100 times
  - → P(exams | students opened their) = 0.1

Should we have discarded the "proctor" context?

Sparsity Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if *"students opened their* $\boldsymbol{w}$*"* never occurred in data? Then $\boldsymbol{w}$ has probability 0!

(Partial) Solution: Add small δ to the count for every $\boldsymbol{w} \in V$. This is called *smoothing*.

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

Sparsity Problem 2

Problem: What if *"students opened their"* never occurred in data? Then we can't calculate probability for *any* $\boldsymbol{w}$!

(Partial) Solution: Just condition on *"opened their"* instead. This is called *backoff*.

Note: Increasing *n* makes sparsity problems *worse.* Typically we can't have *n* bigger than 5.

Storage Problems with n-gram Language Models

Storage: Need to store count for all *n*-grams you saw in the corpus.

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

Increasing *n* or increasing corpus increases model size!

n-gram Language Models in practice

- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop*

Business and financial news

today the _____

get probability distribution

company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039
…	

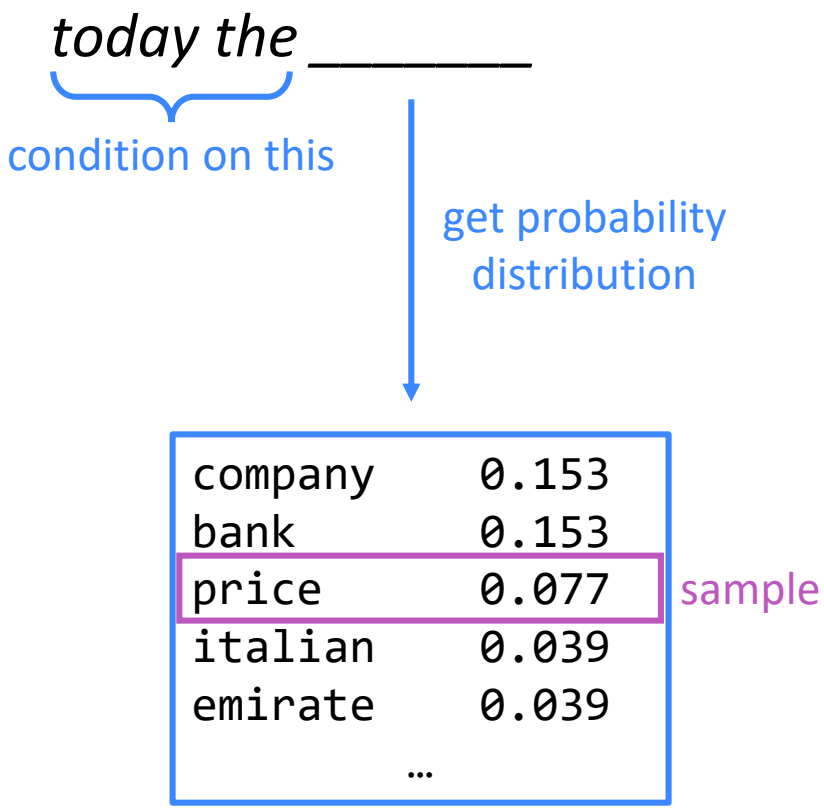
Sparsity problem: not much granularity in the probability distribution

Otherwise, seems reasonable!

*** Try for yourself:** https://nlpforhackers.io/language-models/

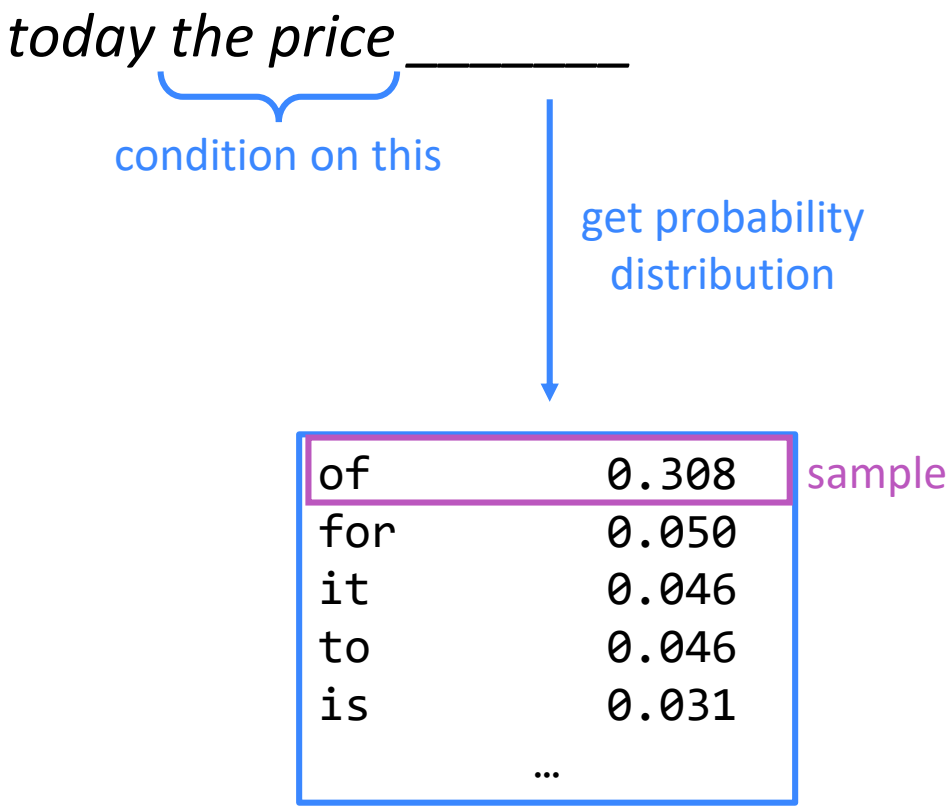
Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.



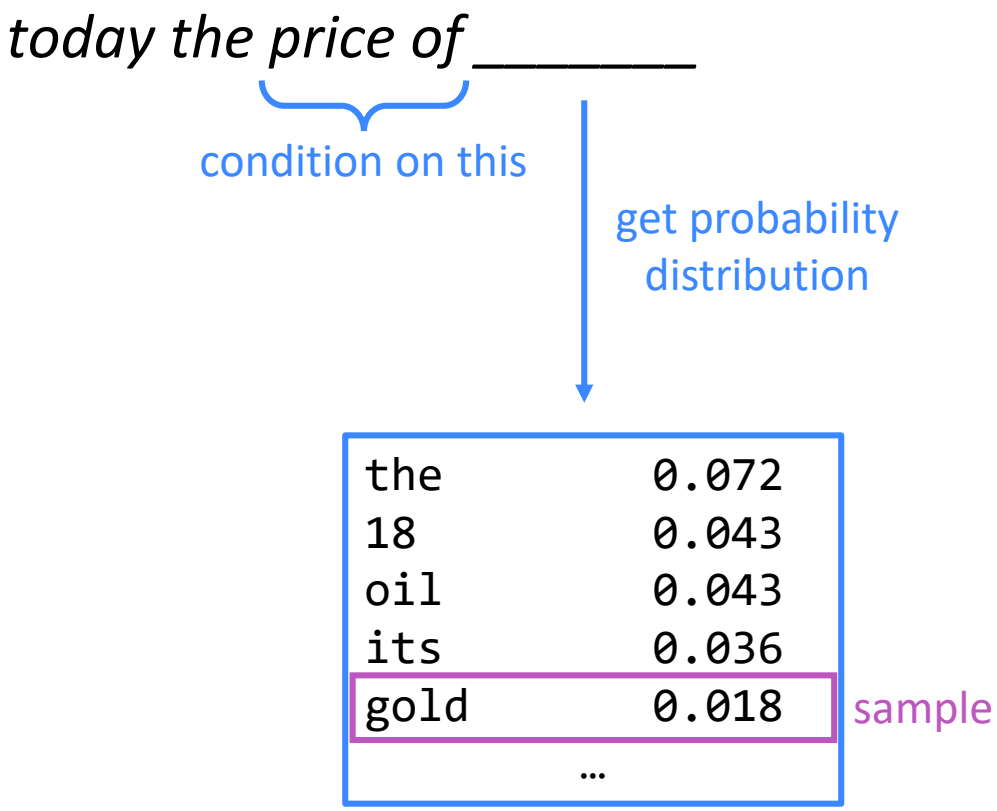
Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.



Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.



Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.

  today the price of gold _____

Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Surprisingly grammatical!

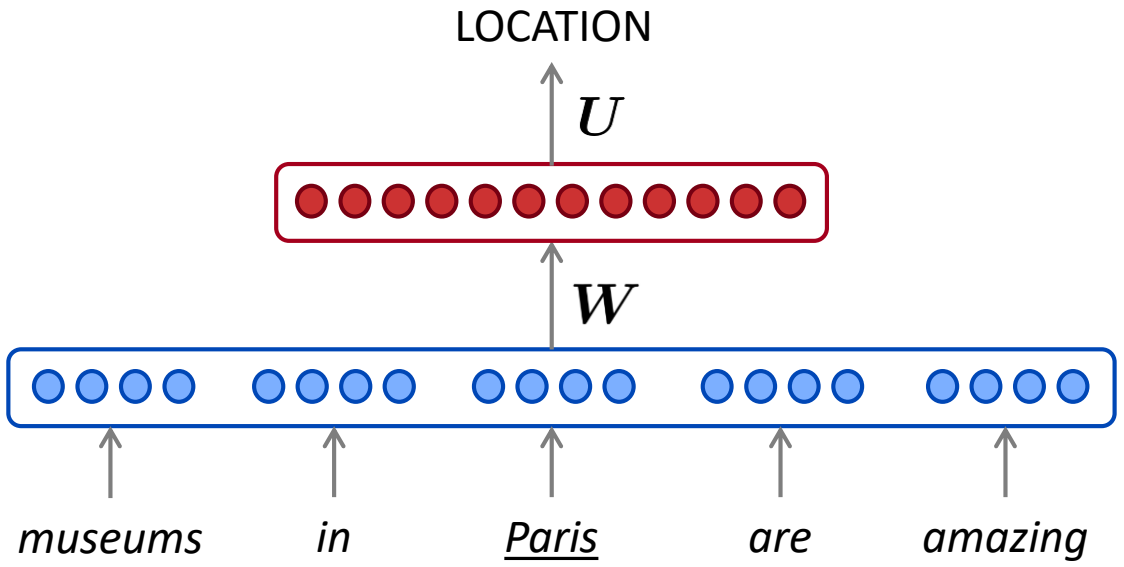
…but **incoherent.** We need to consider more than three words at a time if we want to model language well.

But increasing *n* worsens sparsity problem, and increases model size…

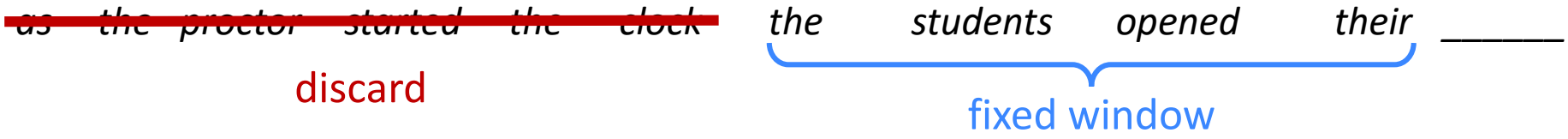
How to build a *neural* Language Model?

- Recall the Language Modeling task:
  - Input: sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
  - Output: prob dist of the next word $P(\boldsymbol{x}^{(t+1)} | \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$

- How about a window-based neural model?
  - We saw this applied to Named Entity Recognition in Lecture 3:



A fixed-window neural Language Model



A fixed-window neural Language Model

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

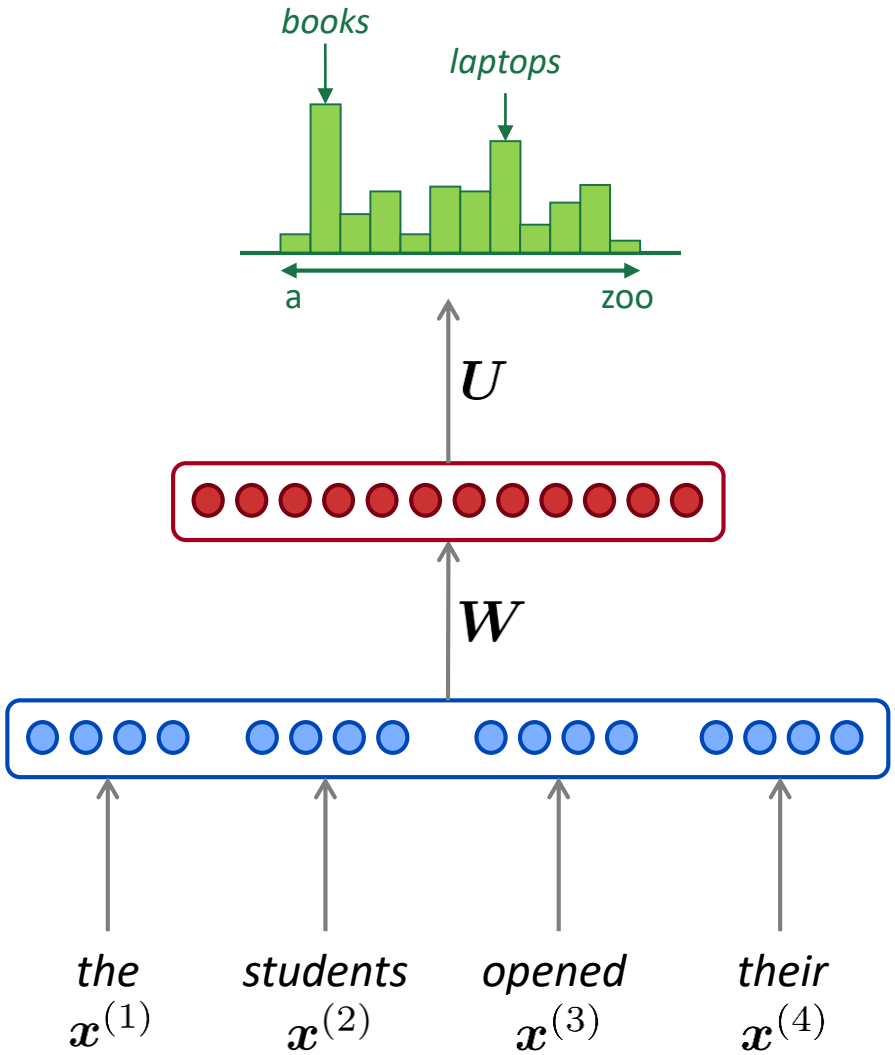
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$



A fixed-window neural Language Model

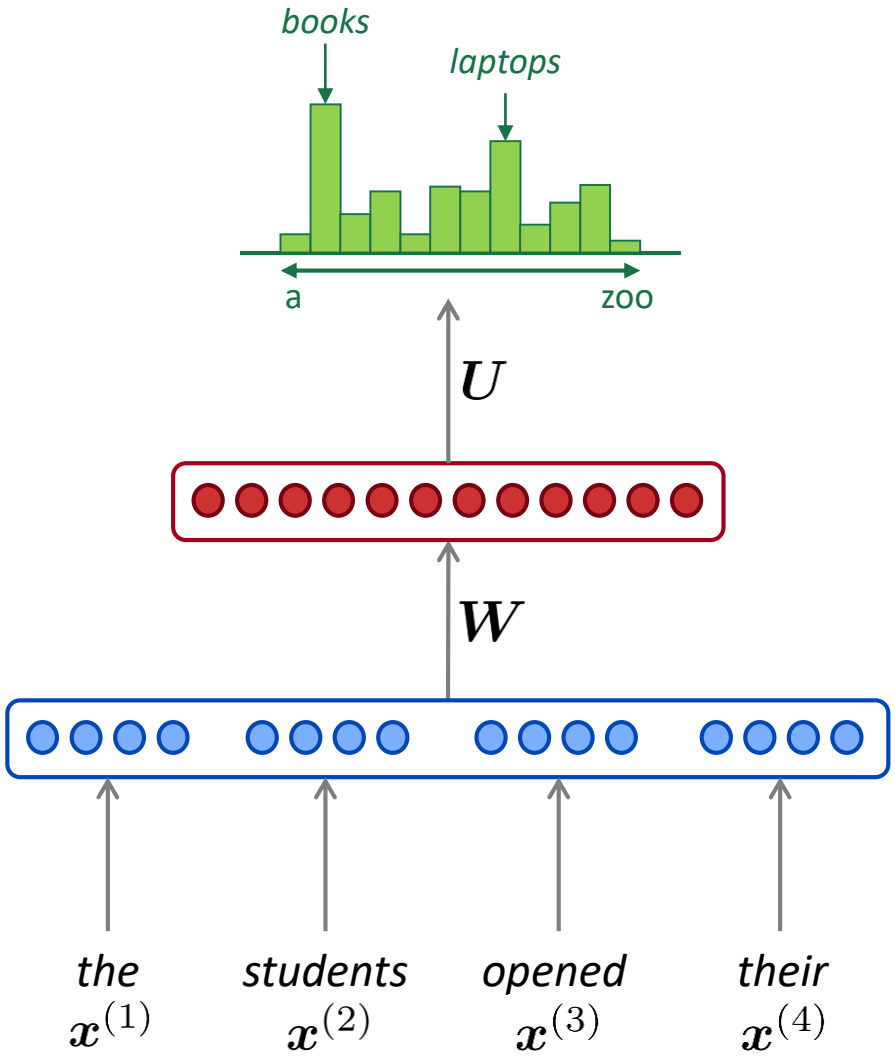
Improvements over *n*-gram LM:

- No sparsity problem
- Don't need to store all observed *n*-grams

Remaining **problems**:

- Fixed window is too small
- Enlarging window enlarges $\boldsymbol{W}$
- Window can never be large enough!
- $\boldsymbol{x}^{(1)}$ and $\boldsymbol{x}^{(2)}$ are multiplied by completely different weights in $\boldsymbol{W}$. No symmetry in how the inputs are processed.

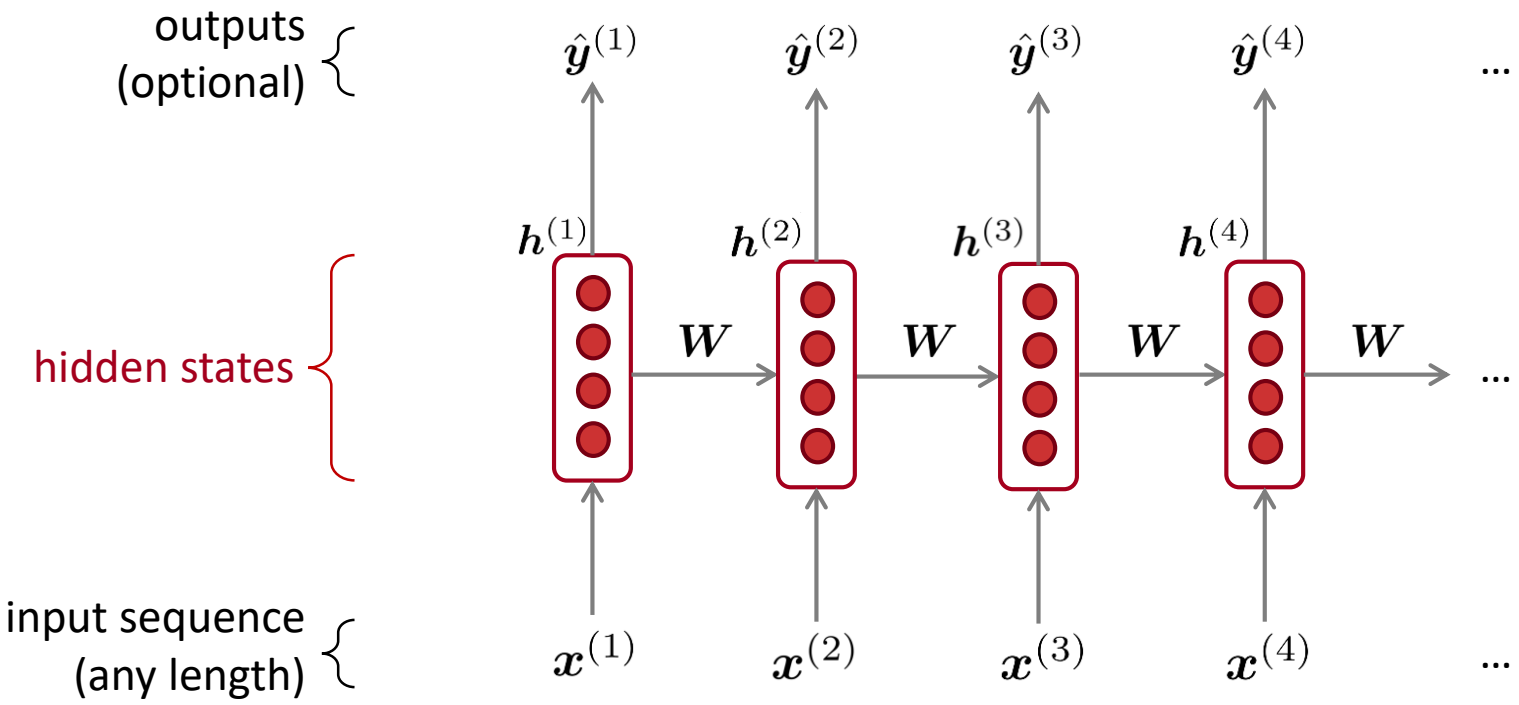
We need a neural architecture that can process *any length input*



Recurrent Neural Networks (RNN)

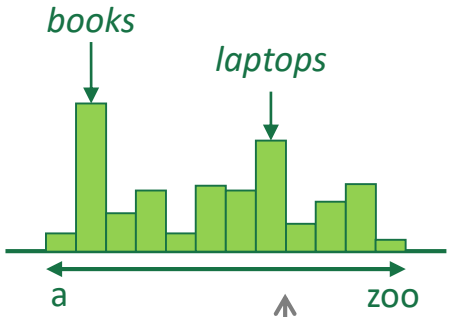
A family of neural architectures

Core idea: Apply the same weights $\boldsymbol{W}$ *repeatedly*



A RNN Language Model

$\hat{\boldsymbol{y}}^{(4)} = P(\boldsymbol{x}^{(5)}|\text{the students opened their})$



output distribution

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}\left(\boldsymbol{U}\boldsymbol{h}^{(t)} + \boldsymbol{b}_2\right) \in \mathbb{R}^{|V|}$$

hidden states

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_e\boldsymbol{e}^{(t)} + \boldsymbol{b}_1\right)$$

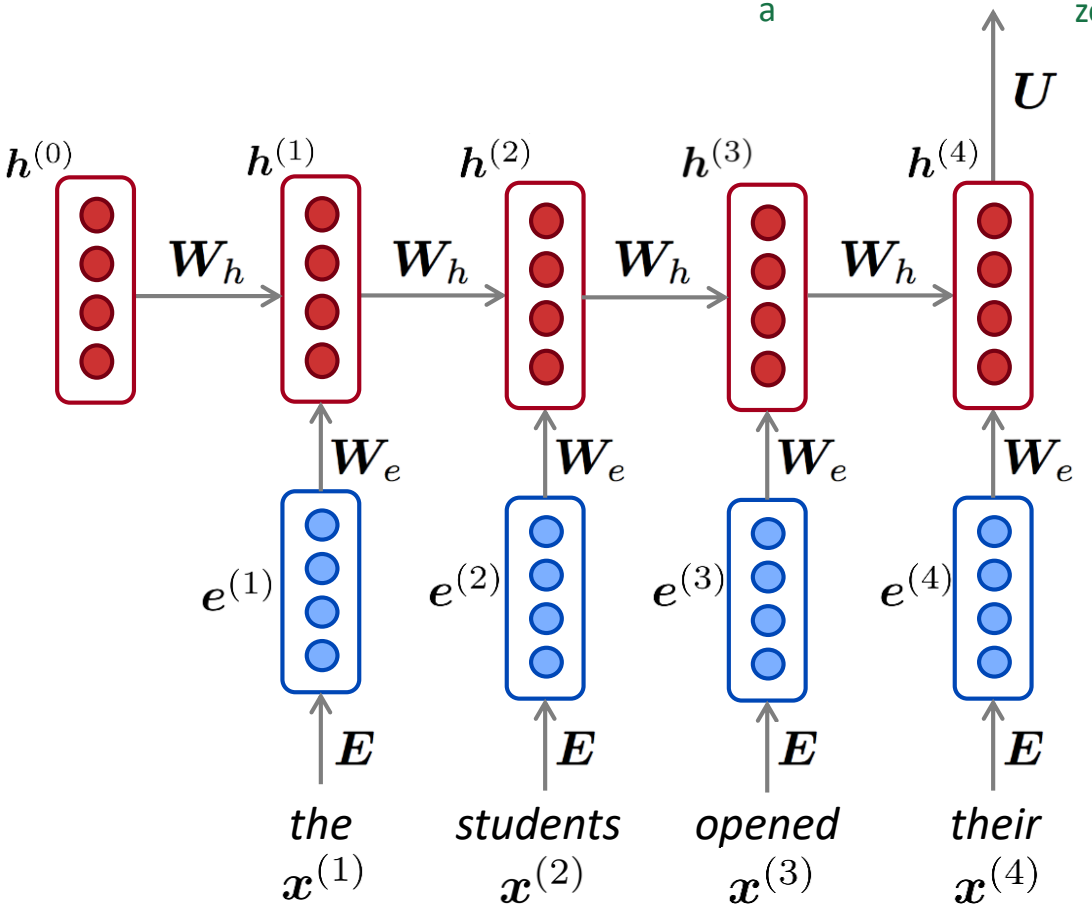
$\boldsymbol{h}^{(0)}$ is the initial hidden state

word embeddings

$$\boldsymbol{e}^{(t)} = \boldsymbol{E}\boldsymbol{x}^{(t)}$$

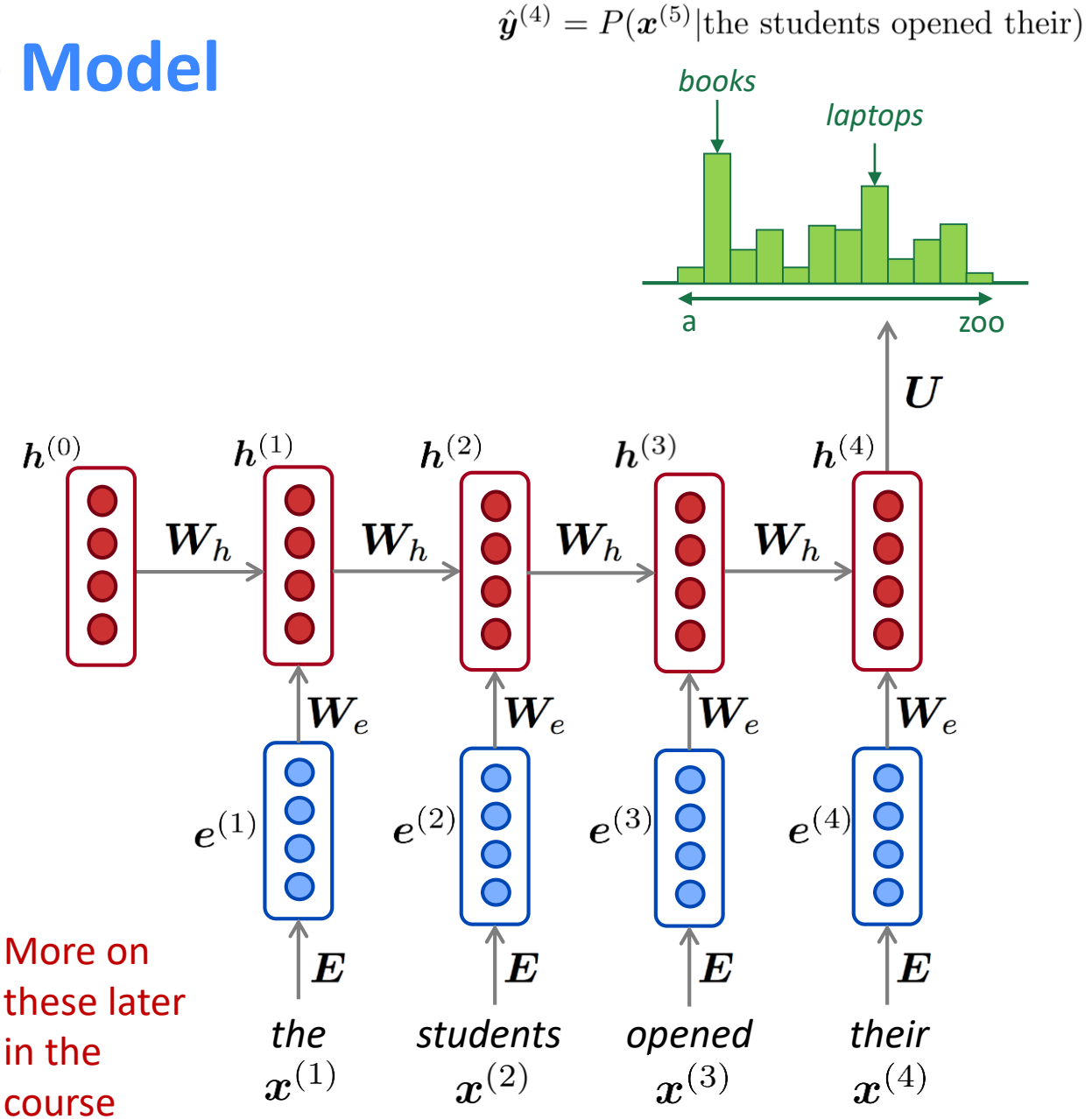
words / one-hot vectors

$$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$$



***Note**: this input sequence could be much longer, but this slide doesn't have space!*

A RNN Language Model



RNN **Advantages**:

- Can process any length input
- Computation for step *t* can (in theory) use information from many steps back
- Model size doesn't increase for longer input
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN **Disadvantages**:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

More on these later in the course

Training a RNN Language Model

- Get a big corpus of text which is a sequence of words $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{\boldsymbol{y}}^{(t)}$ for *every step t*.
  - i.e. predict probability dist of *every word*, given words so far

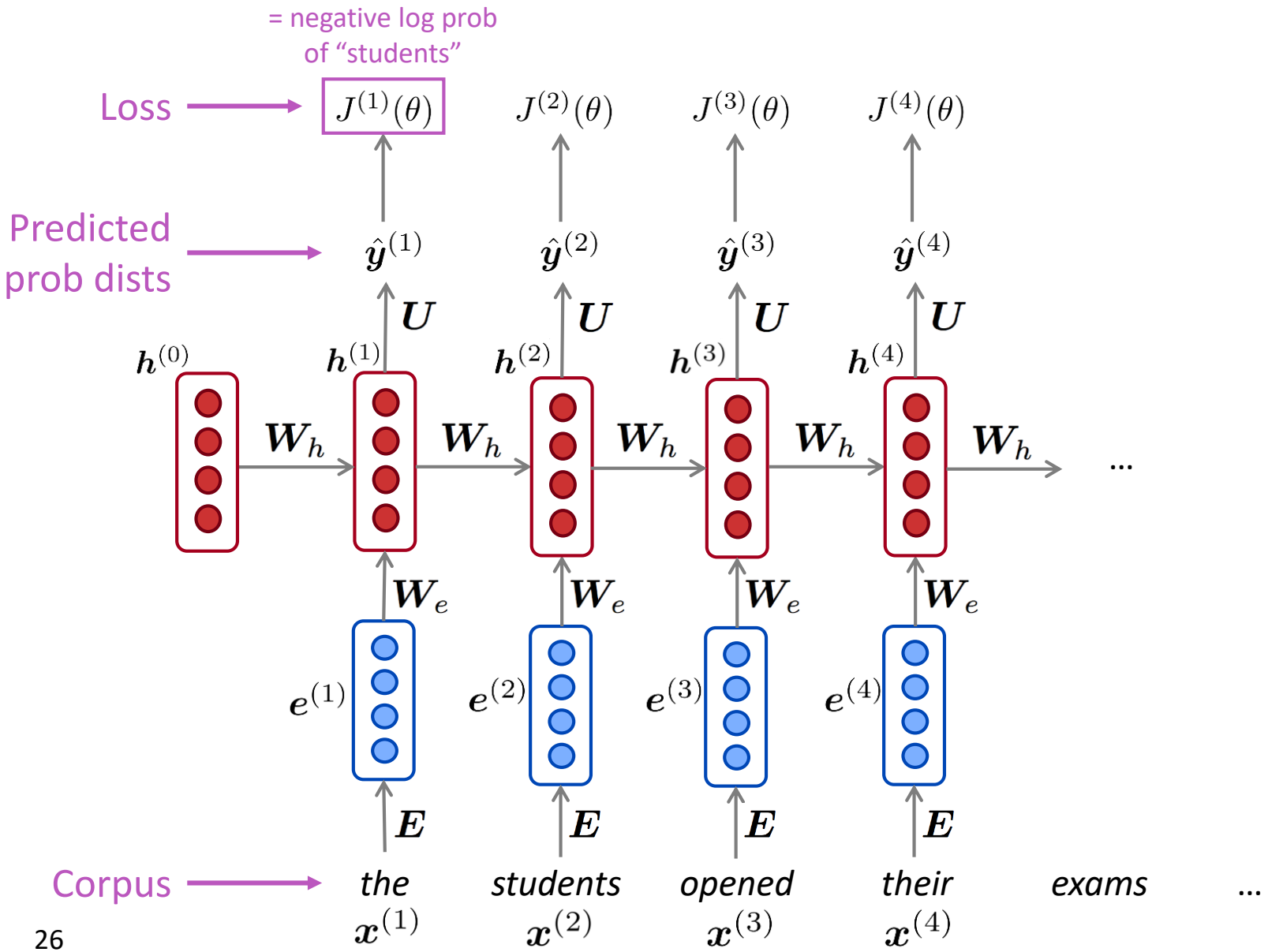
- Loss function on step *t* is cross-entropy between predicted probability distribution $\hat{\boldsymbol{y}}^{(t)}$, and the true next word $\boldsymbol{y}^{(t)}$ (one-hot for $\boldsymbol{x}^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\boldsymbol{y}^{(t)}, \hat{\boldsymbol{y}}^{(t)}) = -\sum_{w \in V} \boldsymbol{y}_w^{(t)} \log \hat{\boldsymbol{y}}_w^{(t)} = -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

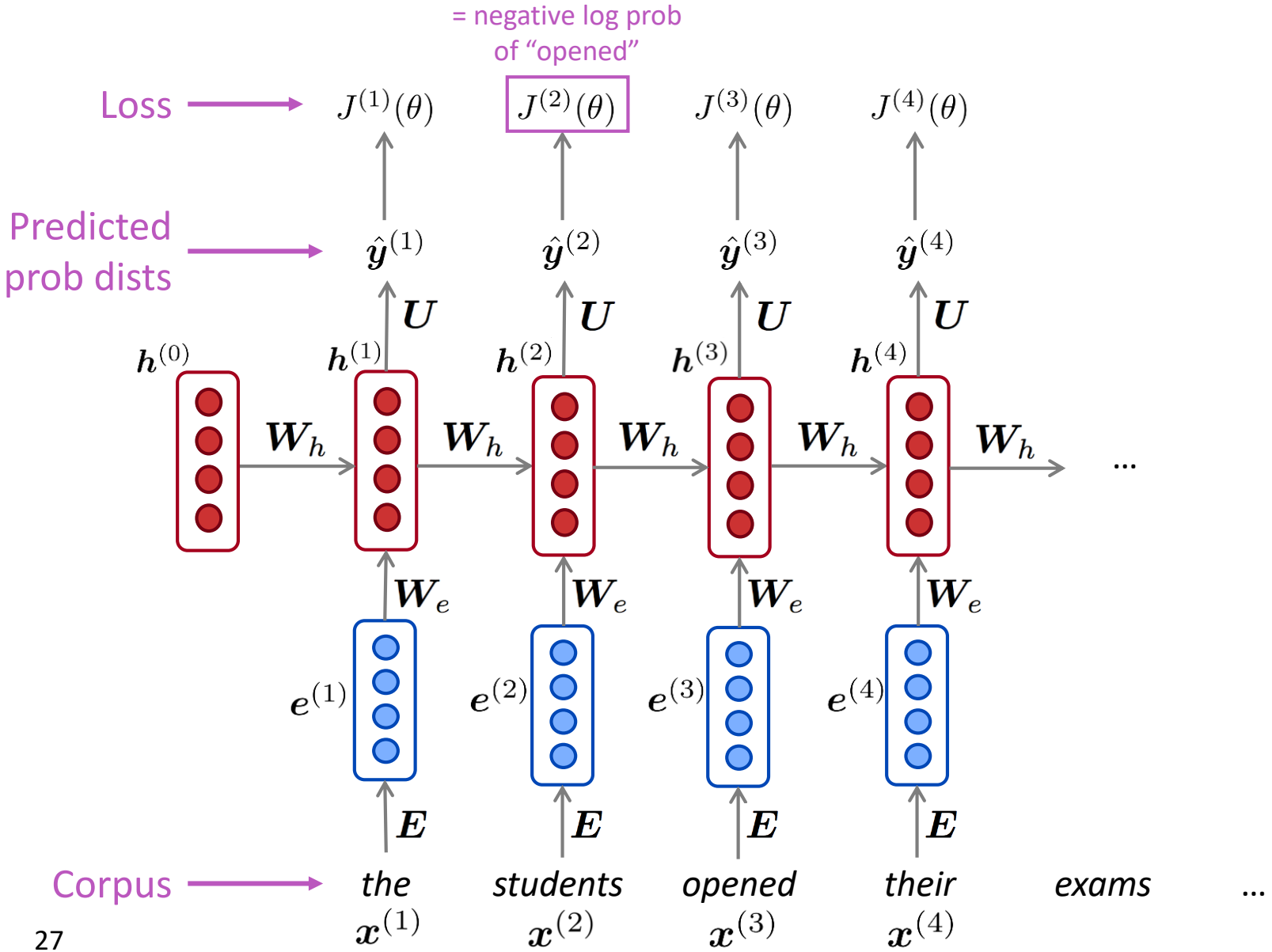
- Average this to get overall loss for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^{T} -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

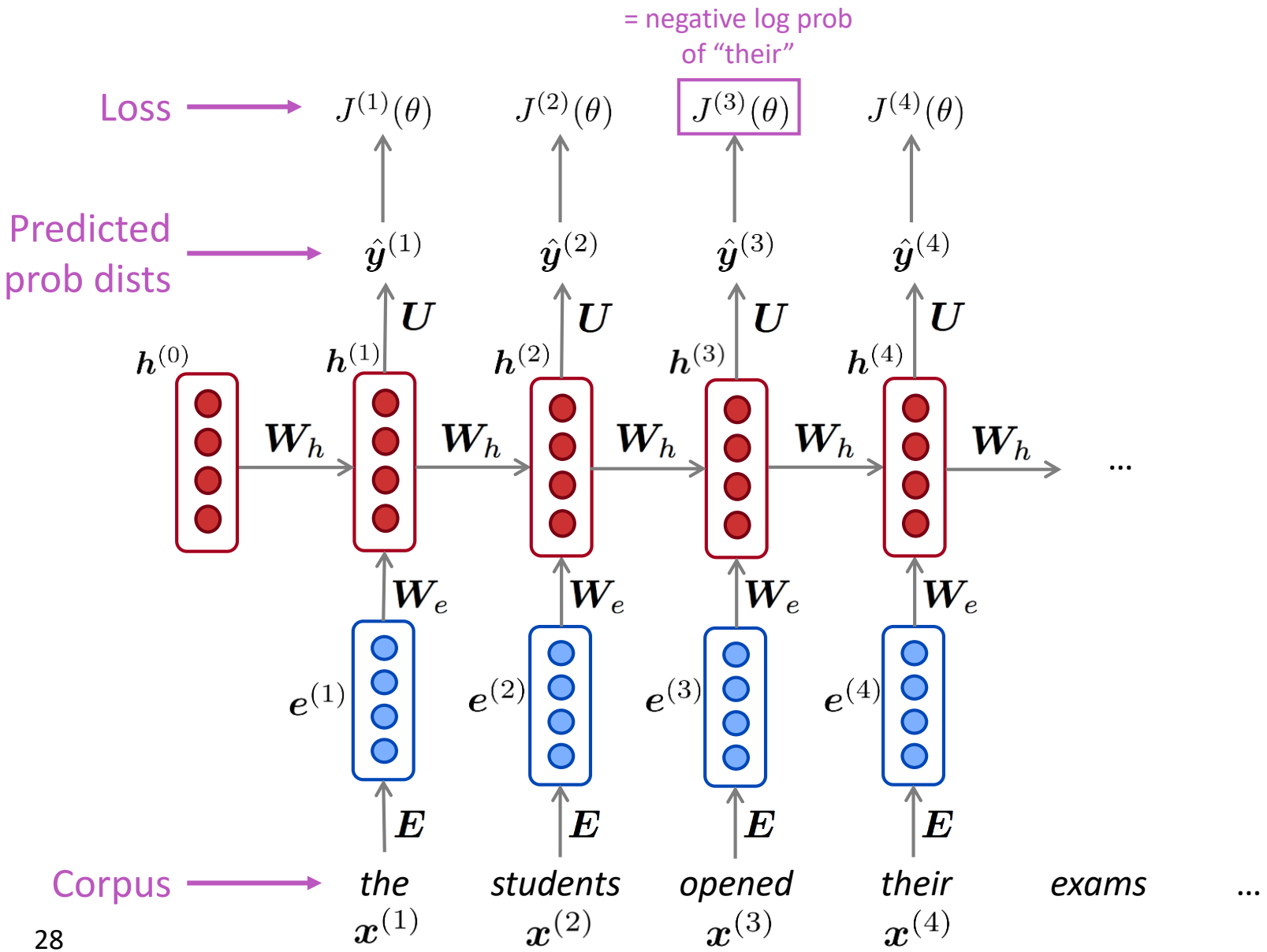
Training a RNN Language Model



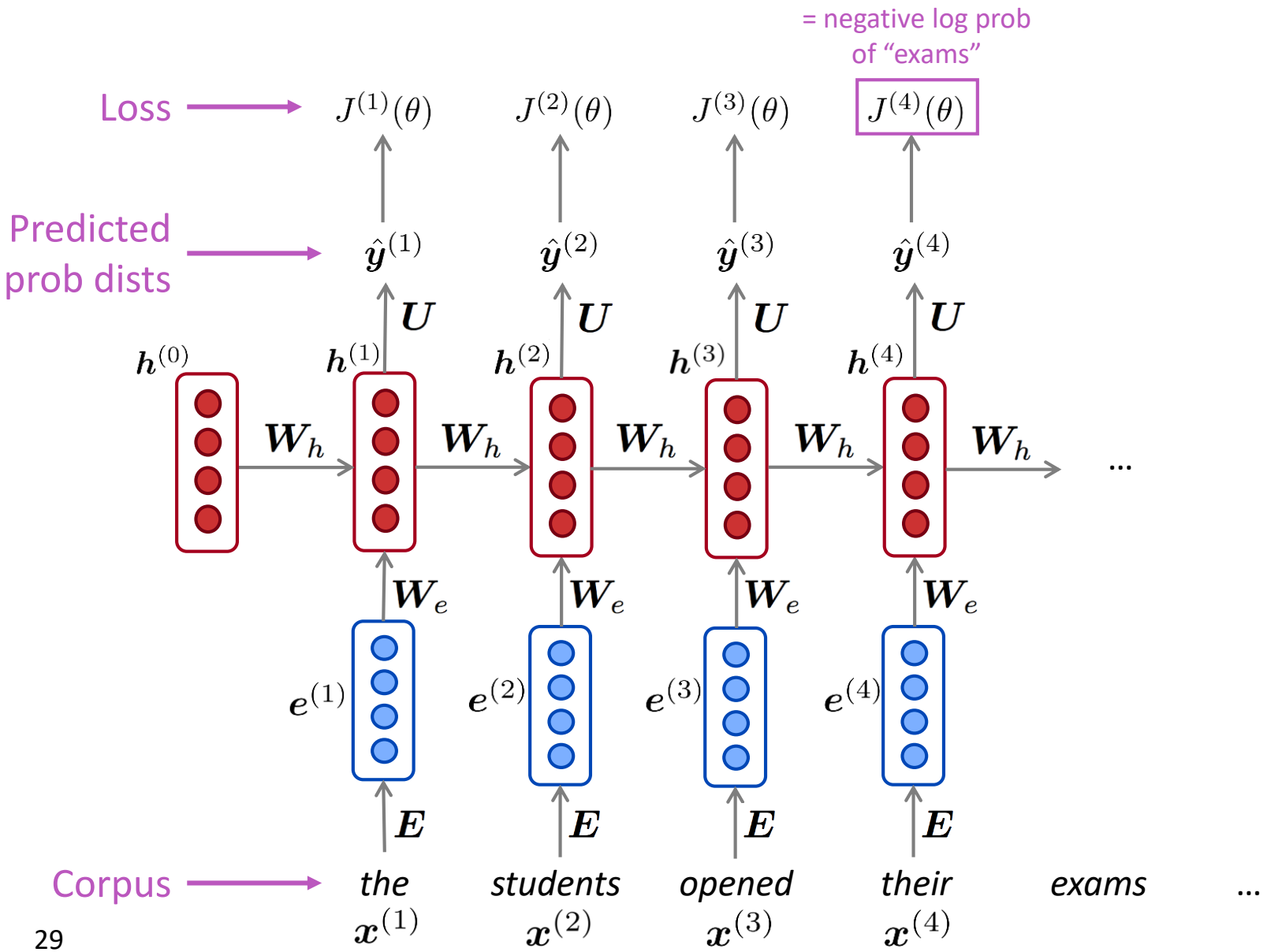
Training a RNN Language Model



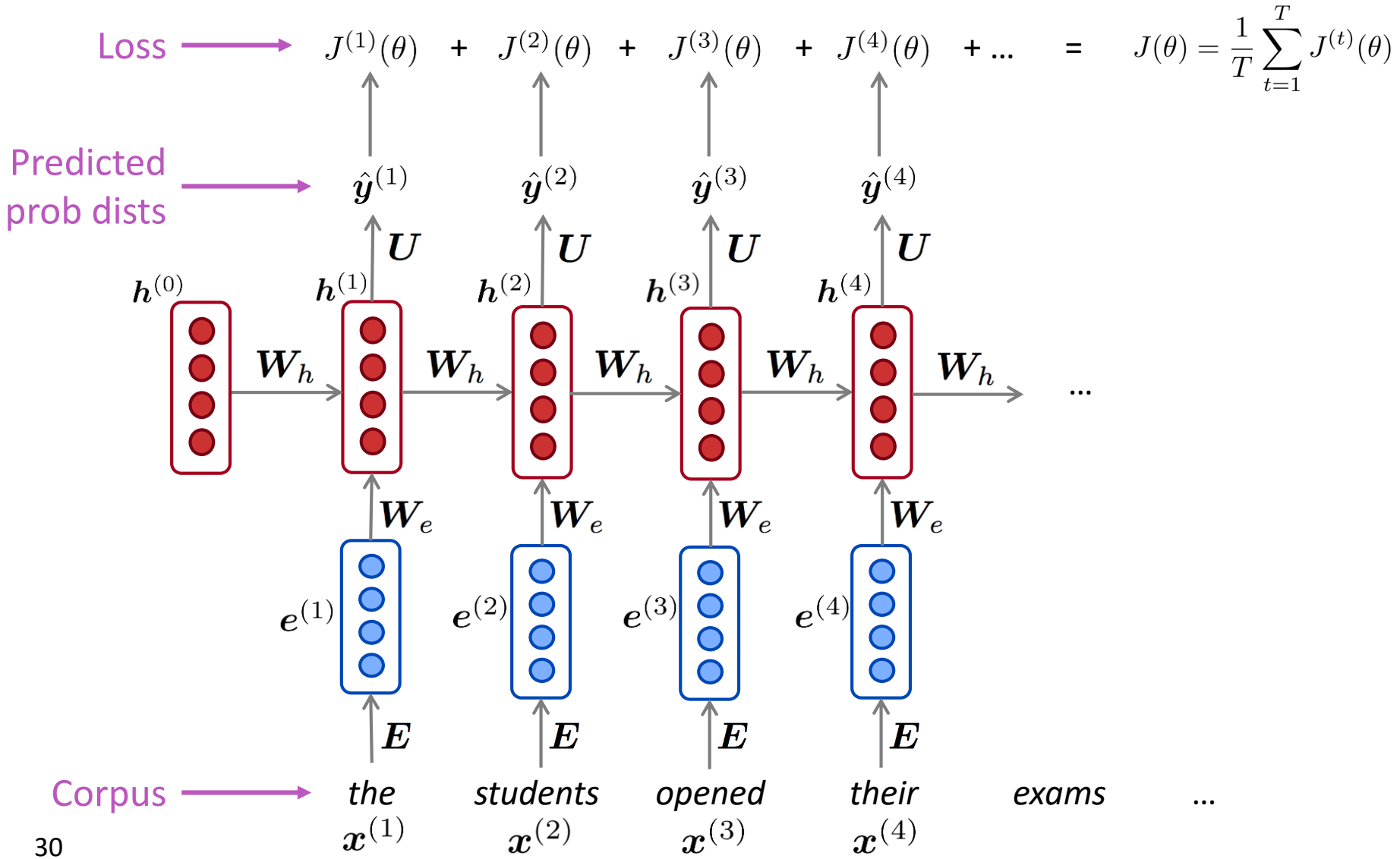
Training a RNN Language Model



Training a RNN Language Model



Training a RNN Language Model



Training a RNN Language Model

- However: Computing loss and gradients across entire corpus $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$ is too expensive!

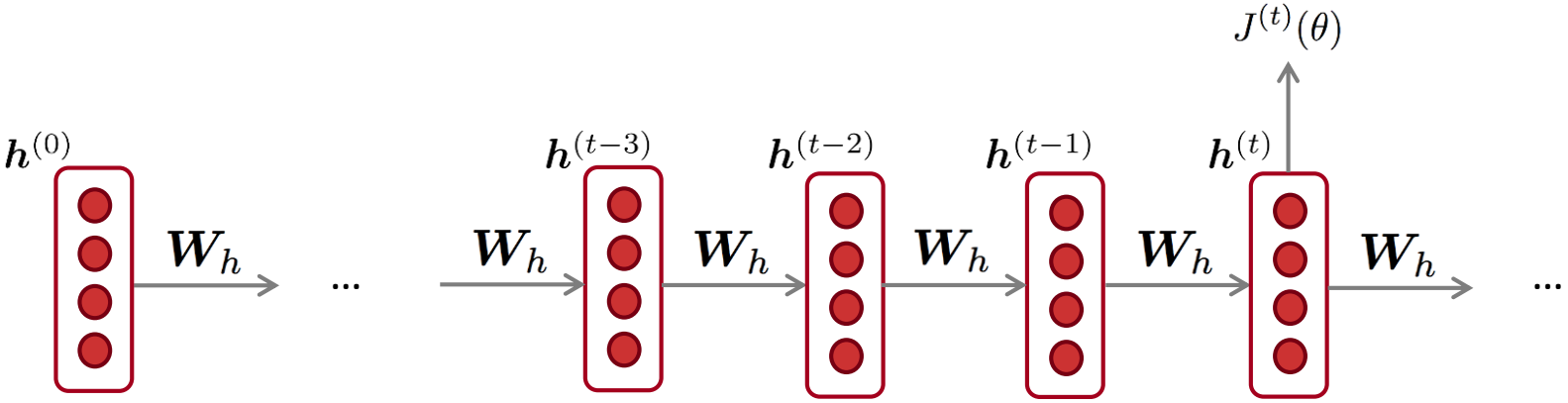
$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta)$$

- In practice, consider $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$ as a sentence (or a document)

- Recall: Stochastic Gradient Descent allows us to compute loss and gradients for small chunk of data, and update.

- Compute loss $J(\theta)$ for a sentence (actually a batch of sentences), compute gradients and update weights. Repeat.

Backpropagation for RNNs



Question: What's the derivative of $J^{(t)}(\theta)$ w.r.t. the repeated weight matrix $\boldsymbol{W}_h$?

Answer:
$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}$$

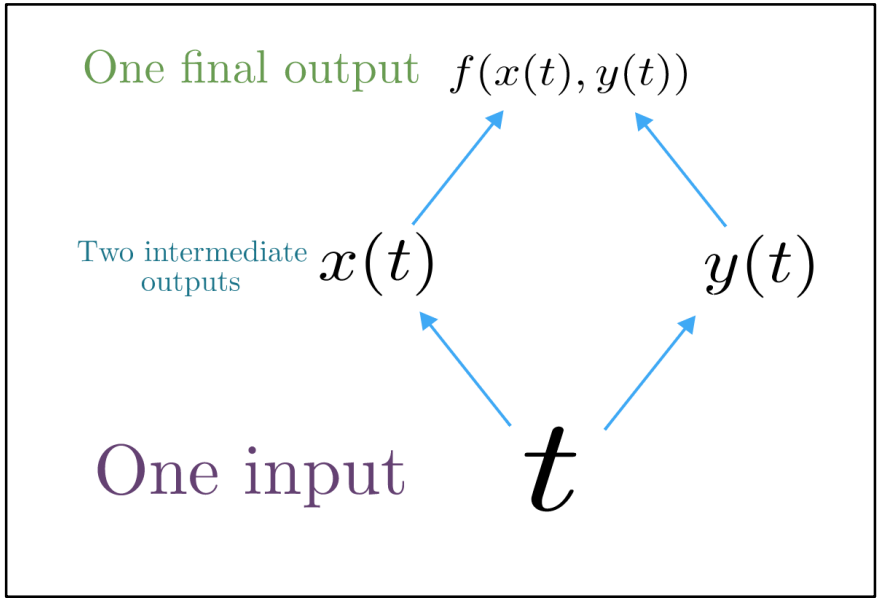
"The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears"

Why?

Multivariable Chain Rule

- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$



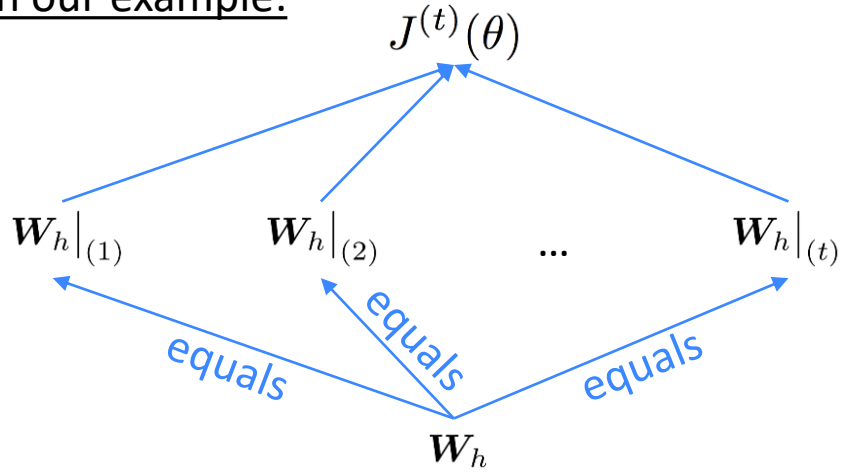
Source:
https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/differentiating-vector-valued-functions/a/multivariable-chain-rule-simple-version

Backpropagation for RNNs: Proof sketch

- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$

In our example:



Apply the multivariable chain rule:

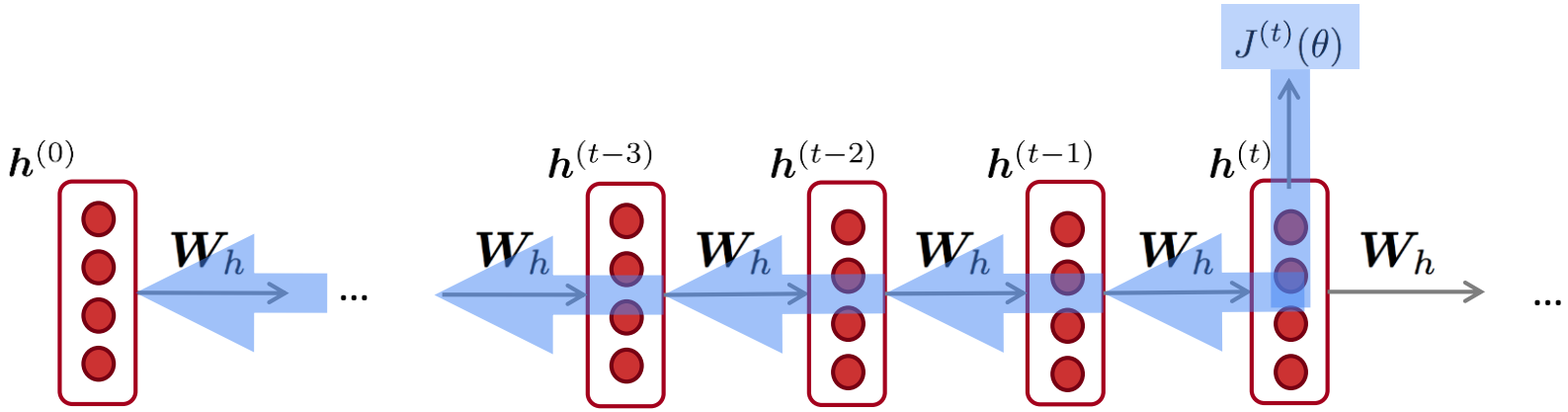
$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h}\bigg|_{(i)} \frac{\partial \boldsymbol{W}_h|_{(i)}}{\partial \boldsymbol{W}_h}$$

$= 1$

$$= \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h}\bigg|_{(i)}$$

Source:
https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/differentiating-vector-valued-functions/a/multivariable-chain-rule-simple-version

Backpropagation for RNNs



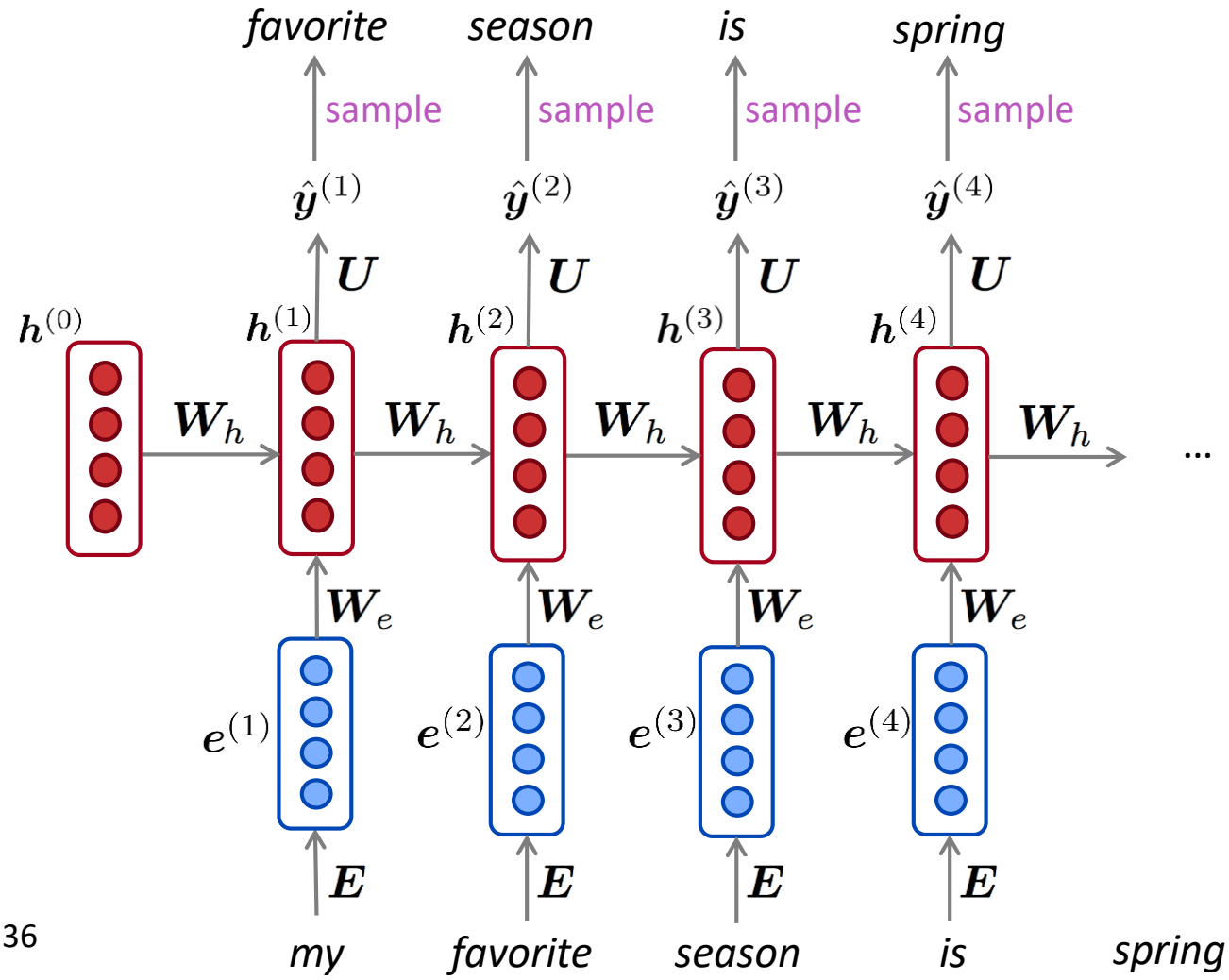
$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\Big|_{(i)}$$

Question: How do we calculate this?

Answer: Backpropagate over timesteps *i*=*t*,…,0, summing gradients as you go.
This algorithm is called **"backpropagation through time"**

Generating text with a RNN Language Model

Just like a n-gram Language Model, you can use a RNN Language Model to generate text by repeated sampling. Sampled output is next step's input.



Generating text with a RNN Language Model

- Let's have some fun!
- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on Obama speeches:



The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem. They were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done.

Source: https://medium.com/@samim/obama-rnn-machine-generated-political-speeches-c8abd18a2ea0

Generating text with a RNN Language Model

- Let's have some fun!
- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on *Harry Potter*:



"Sorry," Harry shouted, panicking—"I'll leave those brooms in London, are they?"

"No idea," said Nearly Headless Nick, casting low close by Cedric, carrying the last bit of treacle Charms, from Harry's shoulder, and to answer him the common room perched upon it, four arms held a shining knob from when the spider hadn't felt it seemed. He reached the teams too.

Source: https://medium.com/deep-writing/harry-potter-written-by-artificial-intelligence-8a9431803da6

Generating text with a RNN Language Model

- Let's have some fun!
- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on recipes:



```
      Title: CHOCOLATE RANCH BARBECUE
Categories: Game, Casseroles, Cookies, Cookies
      Yield: 6 Servings

      2 tb Parmesan cheese -- chopped
      1 c  Coconut milk
      3    Eggs, beaten

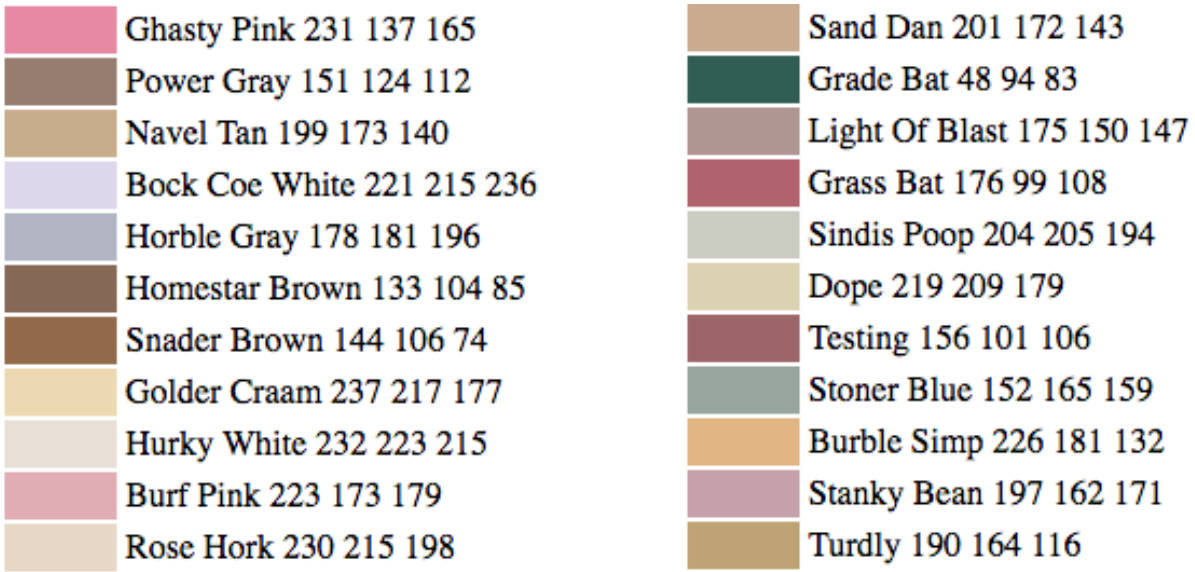
Place each pasta over layers of lumps. Shape mixture into the moderate oven and simmer
until firm. Serve hot in bodied fresh, mustard, orange and cheese.

Combine the cheese and salt together the dough in a large skillet; add the ingredients
and stir in the chocolate and pepper.
```

Source: https://gist.github.com/nylki/1efbaa36635956d35bcc

Generating text with a RNN Language Model

- Let's have some fun!
- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on paint color names:



This is an example of a character-level RNN-LM (predicts what character comes next)

Source: http://aiweirdness.com/post/160776374467/new-paint-colors-invented-by-neural-network

Evaluating Language Models

- The standard evaluation metric for Language Models is perplexity.

$$\text{perplexity} = \prod_{t=1}^{T} \left(\frac{1}{P_{\text{LM}}(\boldsymbol{x}^{(t+1)} | \ \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})} \right)^{1/T}$$

Normalized by number of words

Inverse probability of corpus, according to Language Model

- This is equal to the exponential of the cross-entropy loss $J(\theta)$:

$$= \prod_{t=1}^{T} \left(\frac{1}{\hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}_{t+1}}} \right)^{1/T} = \exp\left(\frac{1}{T} \sum_{t=1}^{T} -\log \hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}_{t+1}} \right) = \exp(J(\theta))$$

Lower perplexity is better!

RNNs have greatly improved perplexity

n-gram model

Increasingly complex RNNs

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

Perplexity improves (lower is better)

Source: https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/

Why should we care about Language Modeling?

- Language Modeling is a benchmark task that helps us measure our progress on understanding language

- Language Modeling is a subcomponent of many NLP tasks, especially those involving generating text or estimating the probability of text:
    - Predictive typing
    - Speech recognition
    - Handwriting recognition
    - Spelling/grammar correction
    - Authorship identification
    - Machine translation
    - Summarization
    - Dialogue
    - etc.

Recap

- Language Model: A system that predicts the next word

- Recurrent Neural Network: A family of neural networks that:
  - Take sequential input of any length
  - Apply the same weights on each step
  - Can optionally produce output on each step

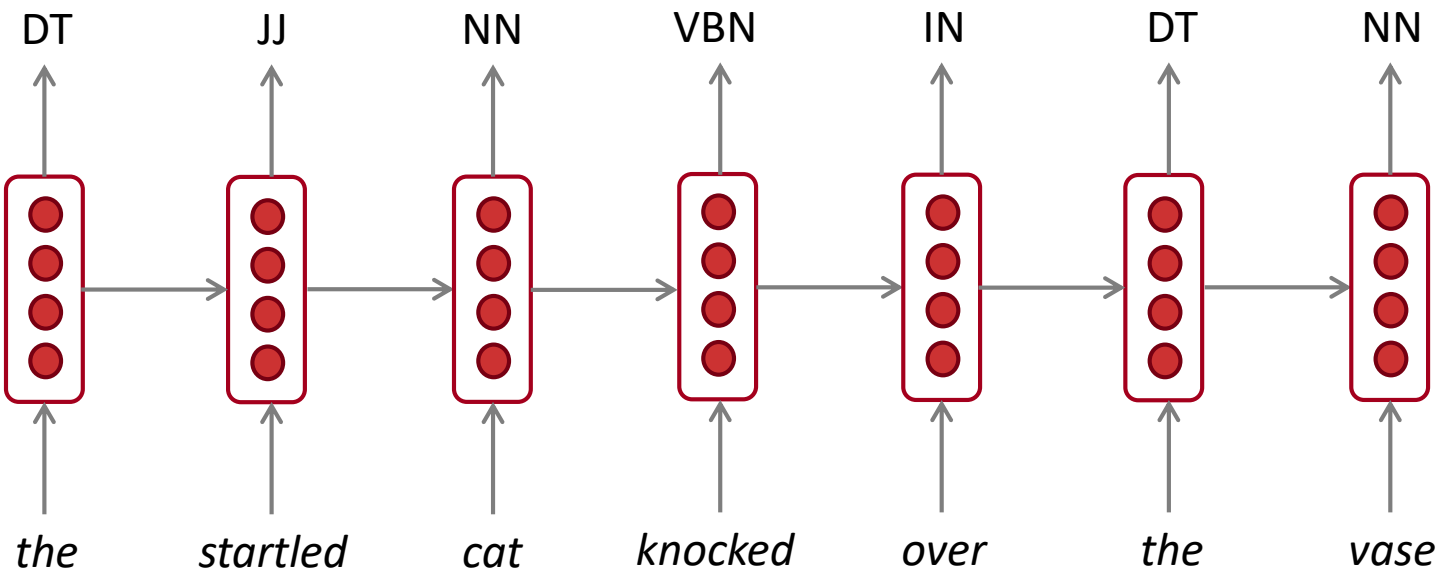
- Recurrent Neural Network ≠ Language Model

- We've shown that RNNs are a great way to build a LM.

- But RNNs are useful for much more!

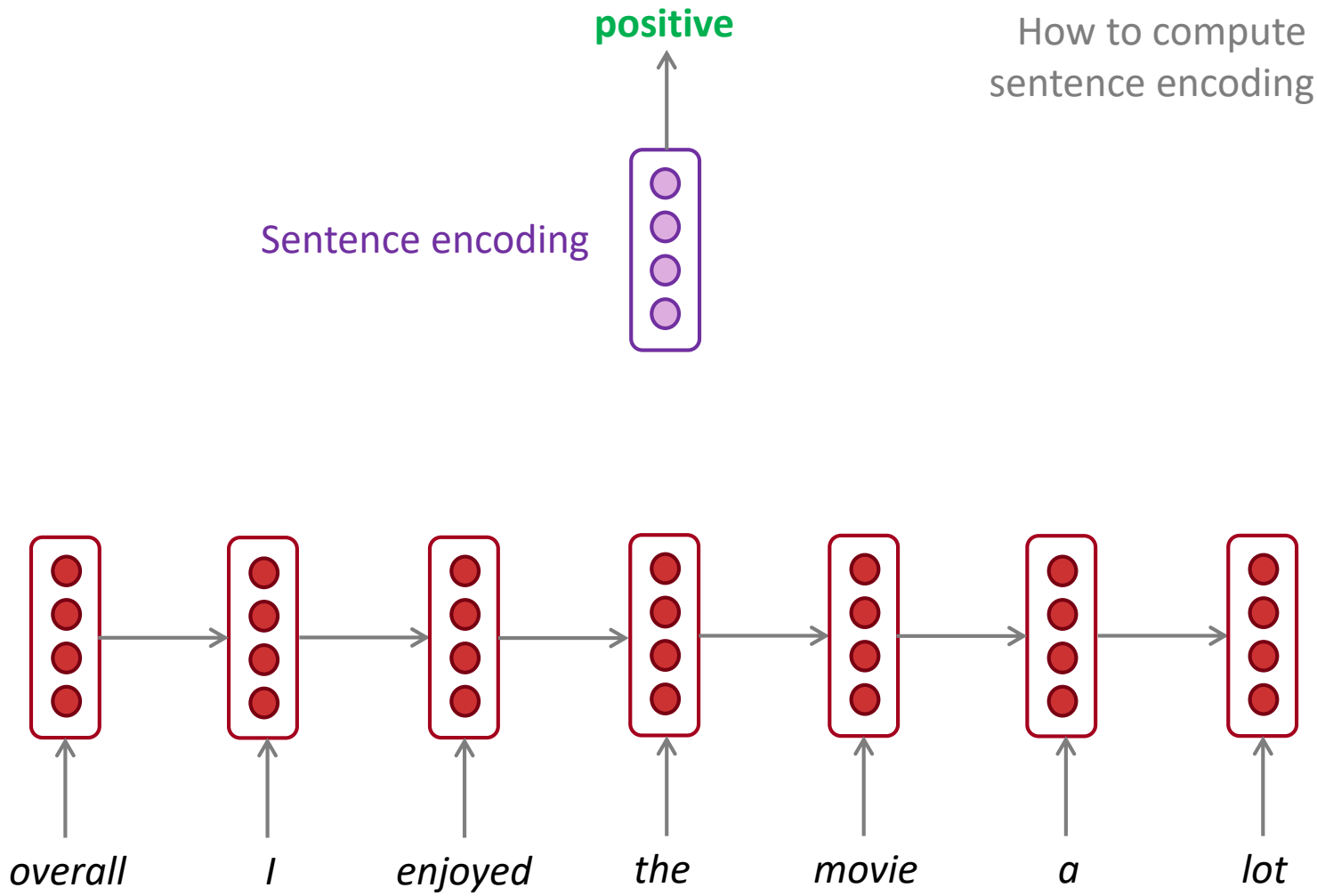
RNNs can be used for tagging

e.g. part-of-speech tagging, named entity recognition



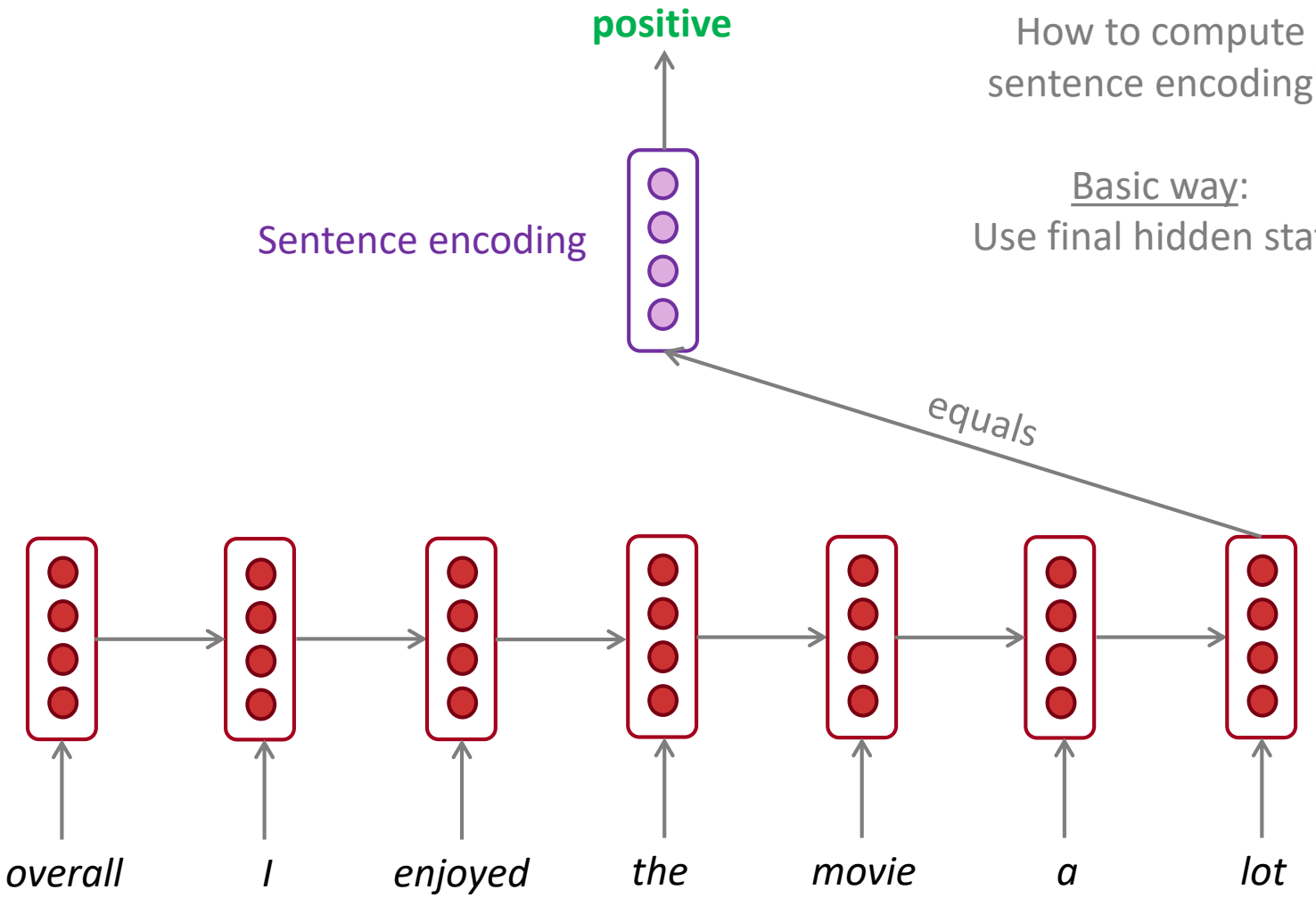
RNNs can be used for sentence classification

e.g. sentiment classification



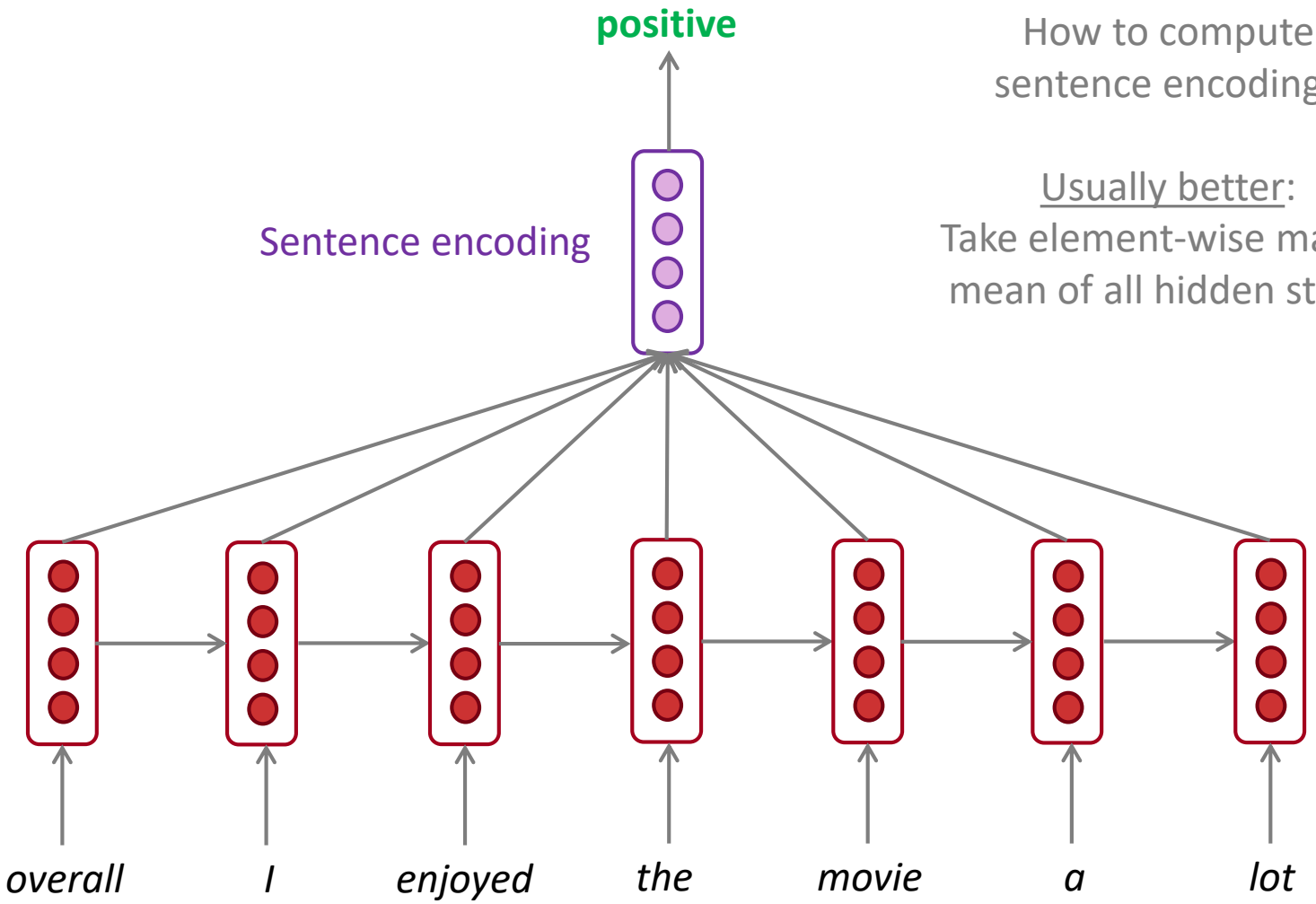
RNNs can be used for sentence classification

e.g. sentiment classification



RNNs can be used for sentence classification

e.g. sentiment classification



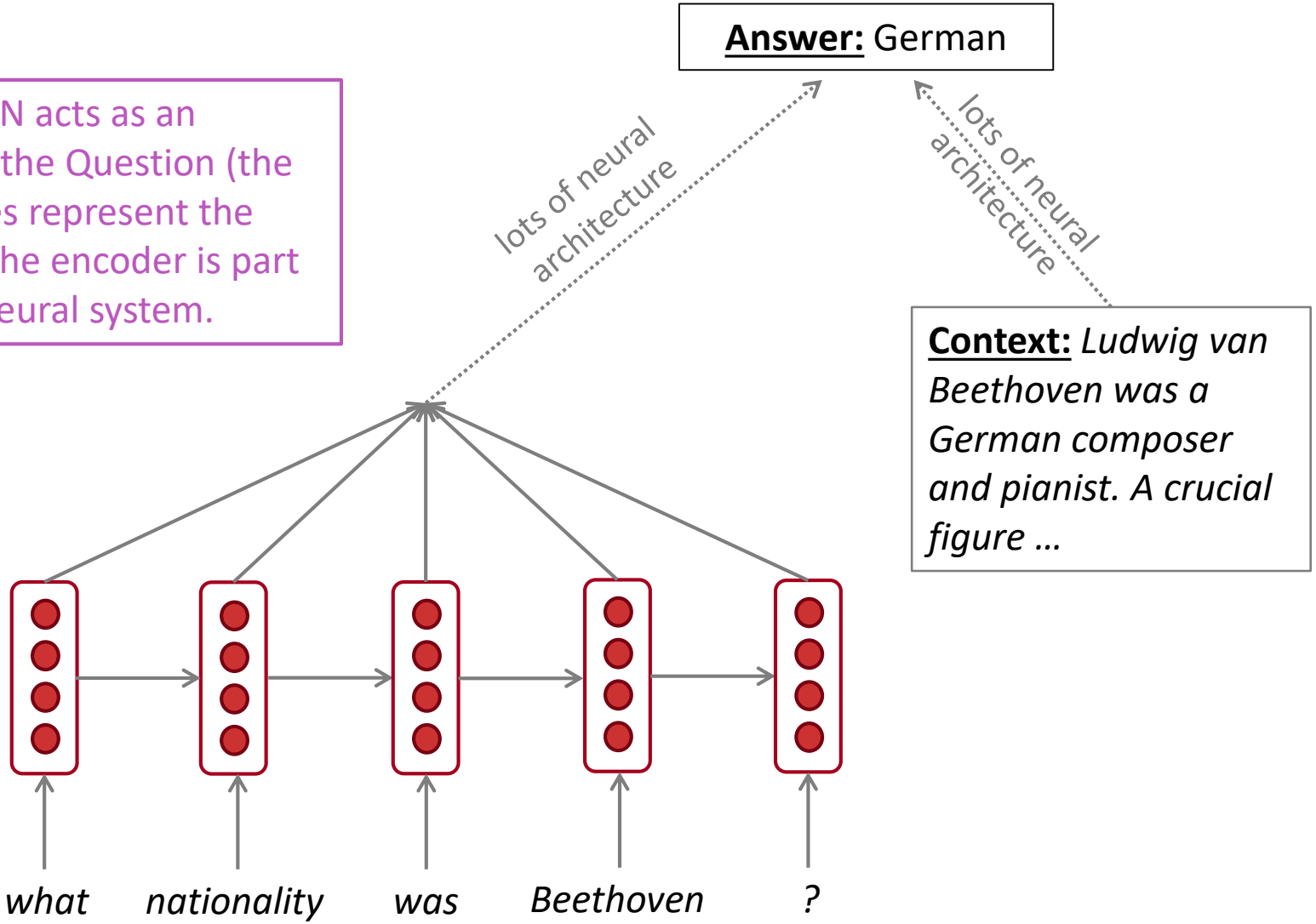
How to compute sentence encoding?

Usually better:
Take element-wise max or mean of all hidden states

RNNs can be used as an encoder module

e.g. question answering, machine translation, *many other tasks!*

Here the RNN acts as an **encoder** for the Question (the hidden states represent the Question). The encoder is part of a larger neural system.



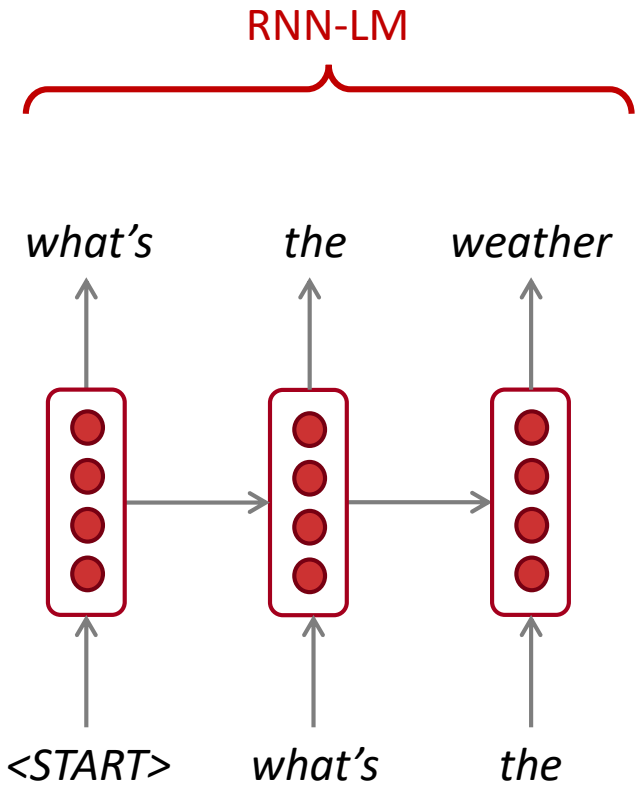
RNN-LMs can be used to generate text

e.g. speech recognition, machine translation, summarization

Input (audio)



conditioning



This is an example of a *conditional language model*.
We'll see Machine Translation in much more detail later.